Documentation BLOG_APP

Application Blog avec CakePHP 5

TERRAL Thomas

BTS SIO – 2025

Sommaire

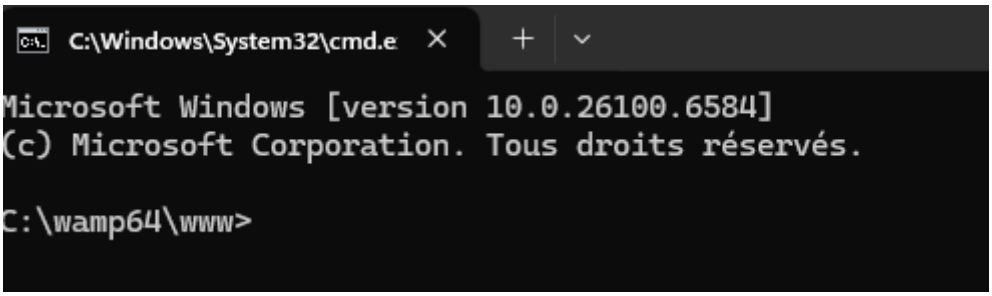
I. Création de la base de données
I.1 Création de projet avec composer
I.2 Importation dans NetBeans
I.3 Créer un utilisateur avec tous les droits pour blog_app uniquement
I.4 Configurer la BDD dans l'app blog_app
II. Migration
II.1 Création migration pour la table article
II.2 Migration pour modifier la table articles
II.3 Migration pour insérer des données dans articles
III. Gestion des Articles
III.1 Création de l'index pour Articles
III.2 Créez la vue pour detail
III.3 Fonction add
III.4 Créer une vue pour add
III.5 Validateur
III.6 Editer un article
III.7 Créez vue edit
IV. Gestion des Users
IV.1 Création de la table User
IV.2 Création des templates de user
V. Changer la page d'accueil de CakePHP
VI. Manipulation et affichage des données
VI.1 Récupération des données
VI.2 Affichage du créateur d'un article
VI.3 Nombre d'articles par user
VII. Gestion des commentaires
VII.1 Clé étrangère
VII.2 Création de la table comments
VII.3 CommentsController
VII.4 Création de la fonction add de commentaire
VII.5 Suppression d'un commentaire
VIII. Authentification
IX. Liens/association entre Users et Comments
IX.1 Ajout des associations
IX.2 Ajout de la clé étrangère user_id
IX.3 Ajout des jointures dans ArticlesController
X. Sécuriser l'intégrité des données
X.1 Disponibilité des boutons edit et supprimer
X.2 Accessibilité de la page edit
X.3 Accessibilité du bouton supprimer des commentaires
XI. Gestion des Tags
XI.1 Création fichier de migration
XI.2 Ajouter nombre de tags dans index article
XI.3 Trier dans l'ordre décroissant les articles modifiés

XI.4 Afficher le nombre d'article pour chaque tag
XI.5 Ajouter les tags dans le add
XI.6 Ajouter les tags dans le edit
XI.7 Afficher les tags dans le detail des articles
XI.8 Supprimer un tag d'un article
XI.9 Afficher si des tags sont associées ou non
XII. Diagramme de classe

I – Création de la base de données

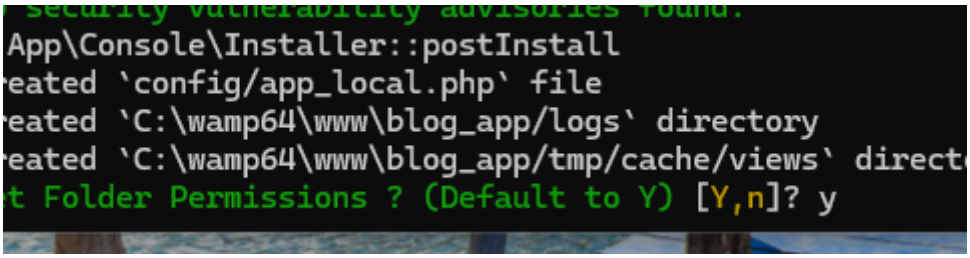
I.1 – Création de projet avec composer

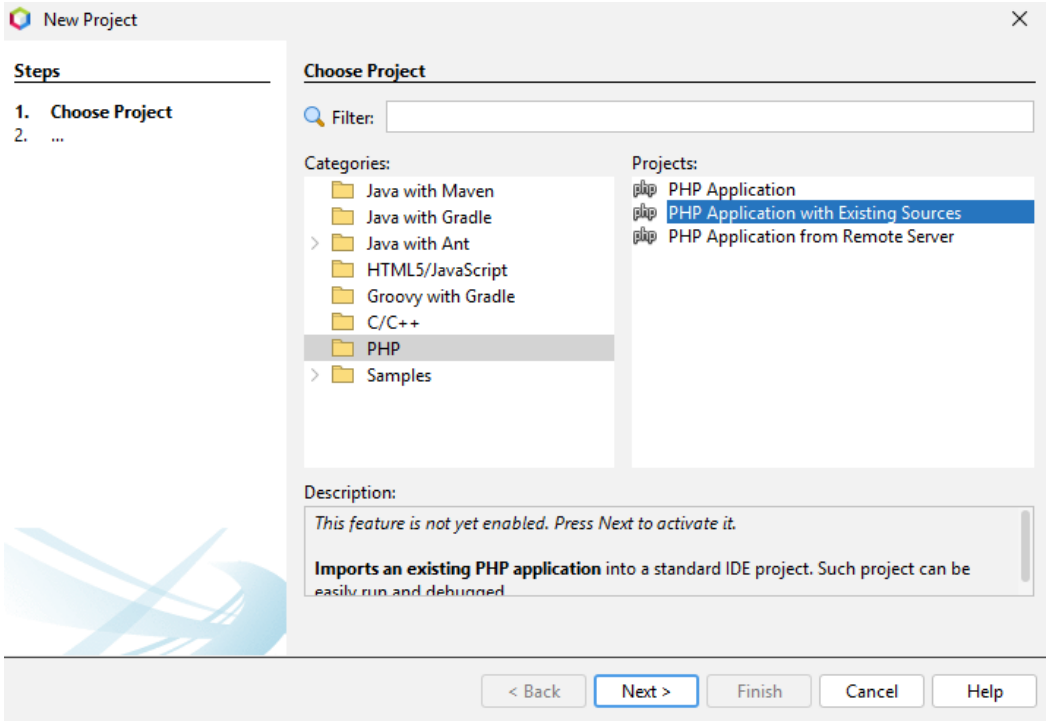
- Accéder au disque C, puis Wamp64 > www, et ouvrir CMD dans la barre d'adresse




```
C:\wamp64\www>composer create-project --prefer-dist cakephp/app:5.* blog_app
Creating a "cakephp/app:5.*" project at "./blog_app"
Installing cakephp/app (5.2.0)
  - Installing cakephp/app (5.2.0): Extracting archive
Created project in C:\wamp64\www\blog_app
Loading composer repositories with package information
Updating dependencies
```

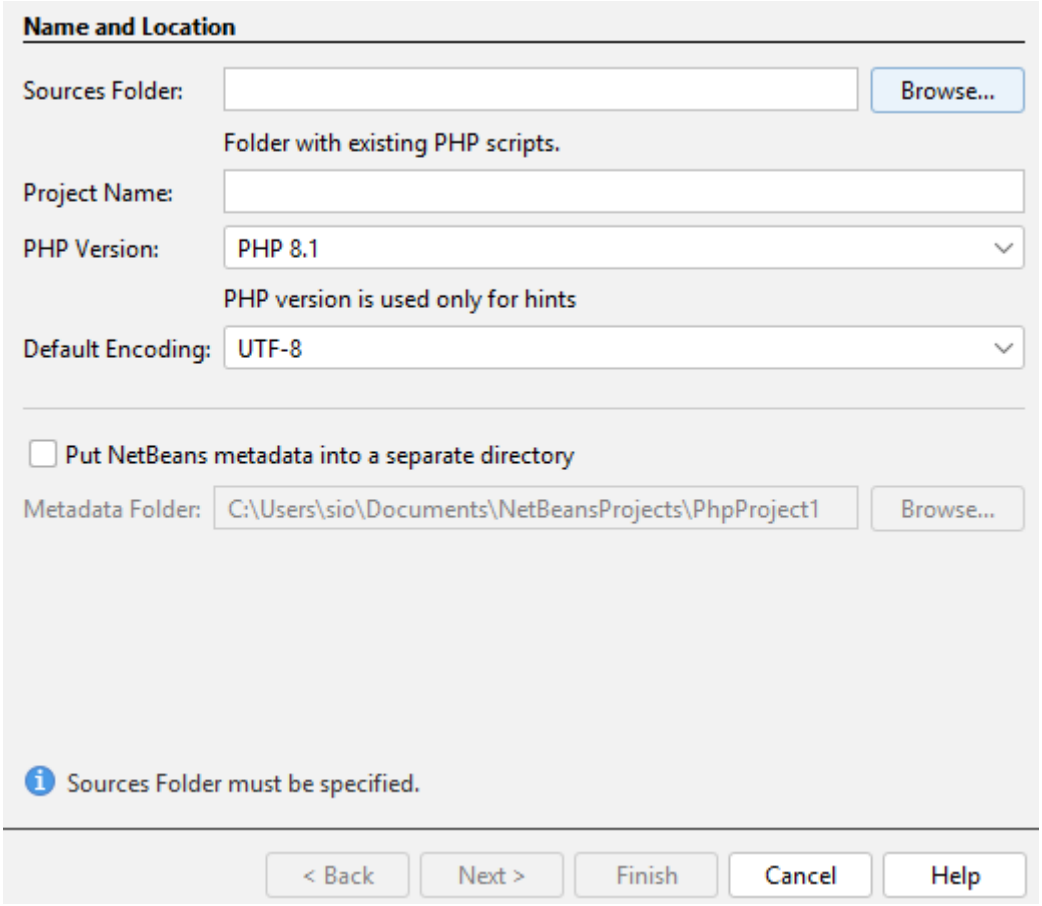
```
C:\wamp64\www>composer create-project --prefer-dist cakephp/app:5.* blog_app
```

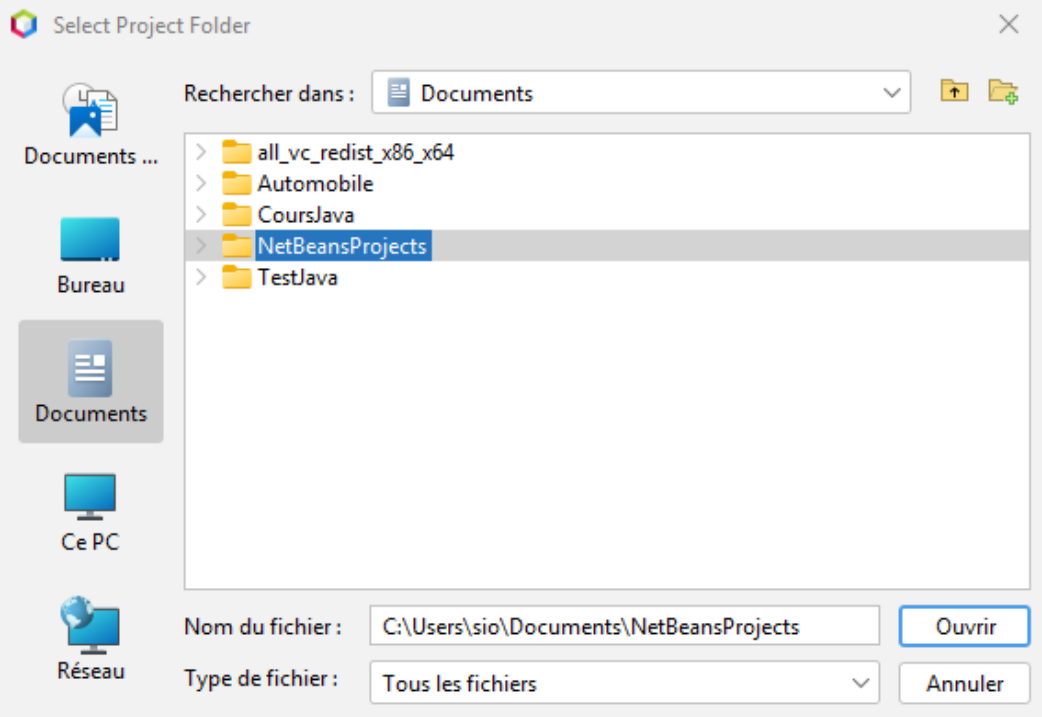




I.2 – Importation dans NetBeans

- Ouvrir NetBeans > File > New Projects > PHP > PHP Application with Existing Sources

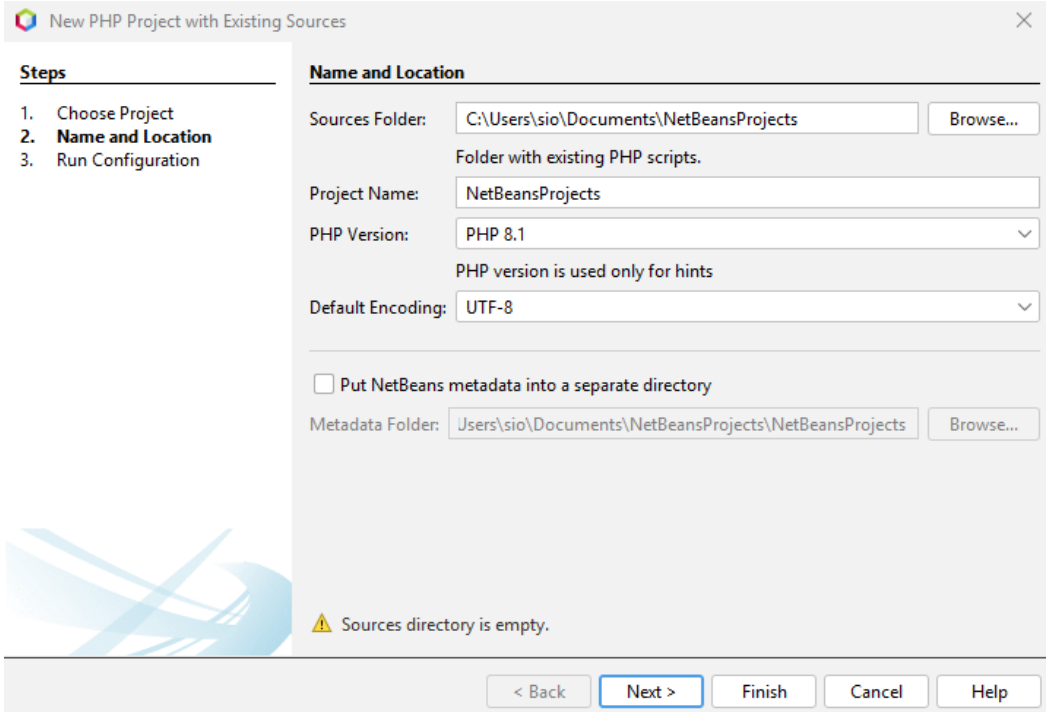


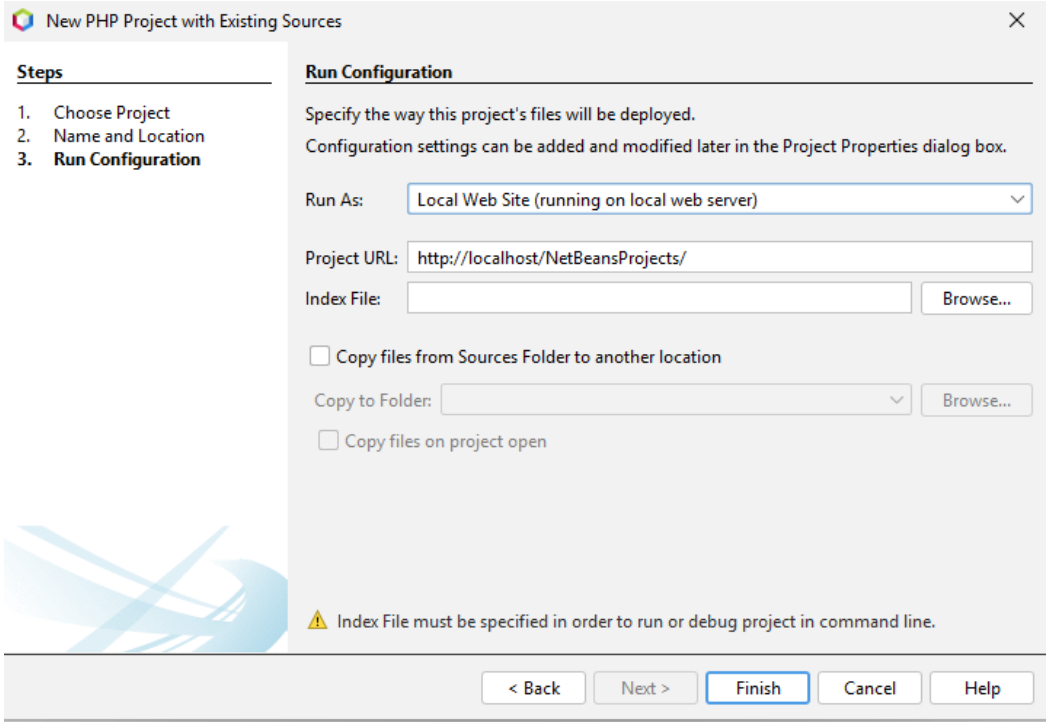


- Cliquer Browse, choisir NetBeansProjects, puis Finish

I.3 – Créer un utilisateur avec tous les droits pour blog_app uniquement

- Naviguer dans C:\wamp64\bin\mysql\mysql9.1.0\bin et ouvrir CMD
- Se connecter en tant que root à MySQL

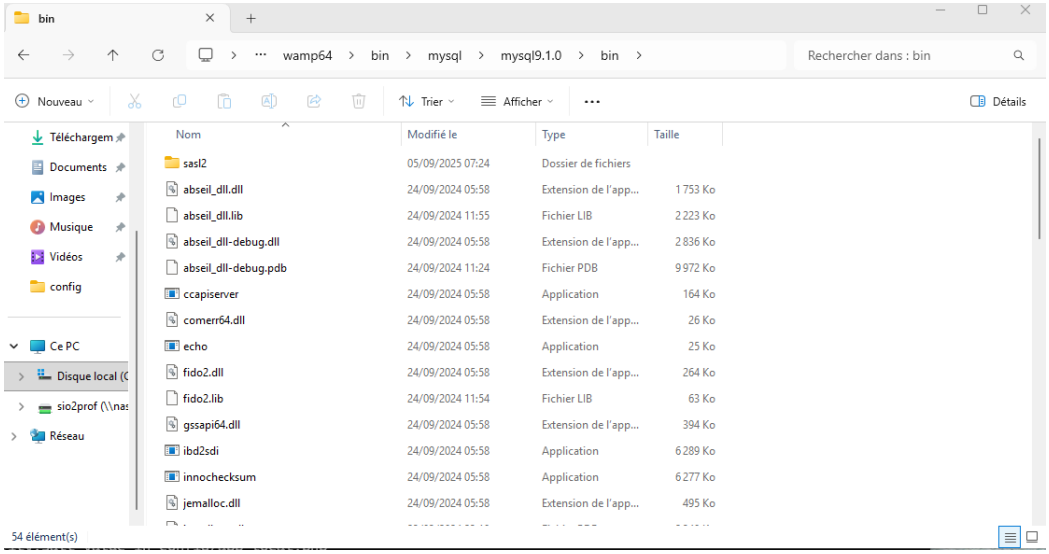




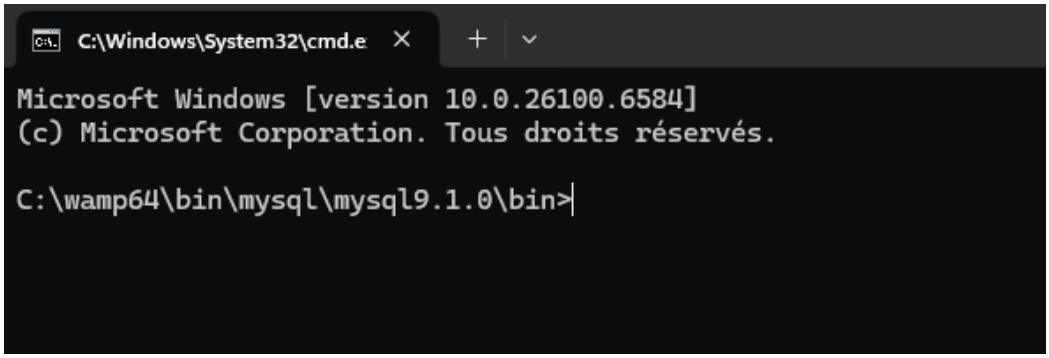

```
mysql> create database blog_app_bdd;
mysql> create user blog_app identified by "Xddl14*";
mysql> grant all privileges on blog_app_bdd.* to blog_app;
```

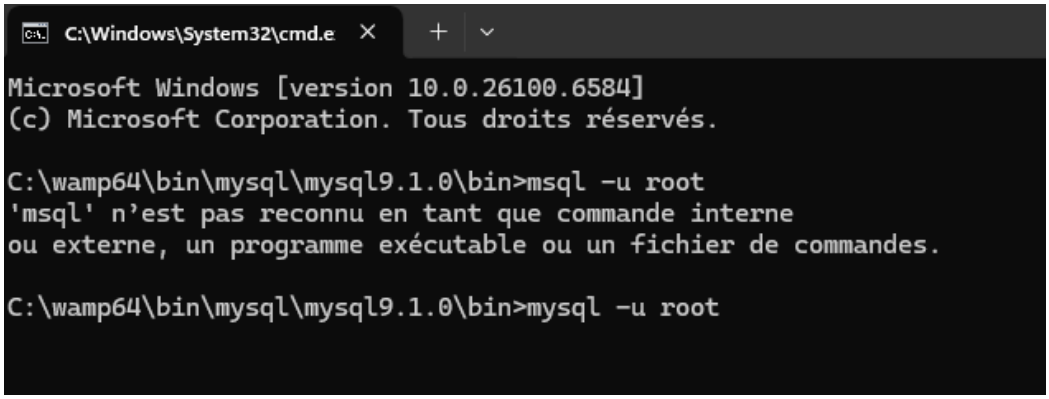
I.4 – Configurer la BDD dans l'app blog_app

- Dans le dossier config, ouvrir app_local.php et modifier les lignes 50, 51 et 53 :




```
'username' => 'blog_app',
'password' => 'Xdd114*',
'database' => 'blog_app_bdd',
```



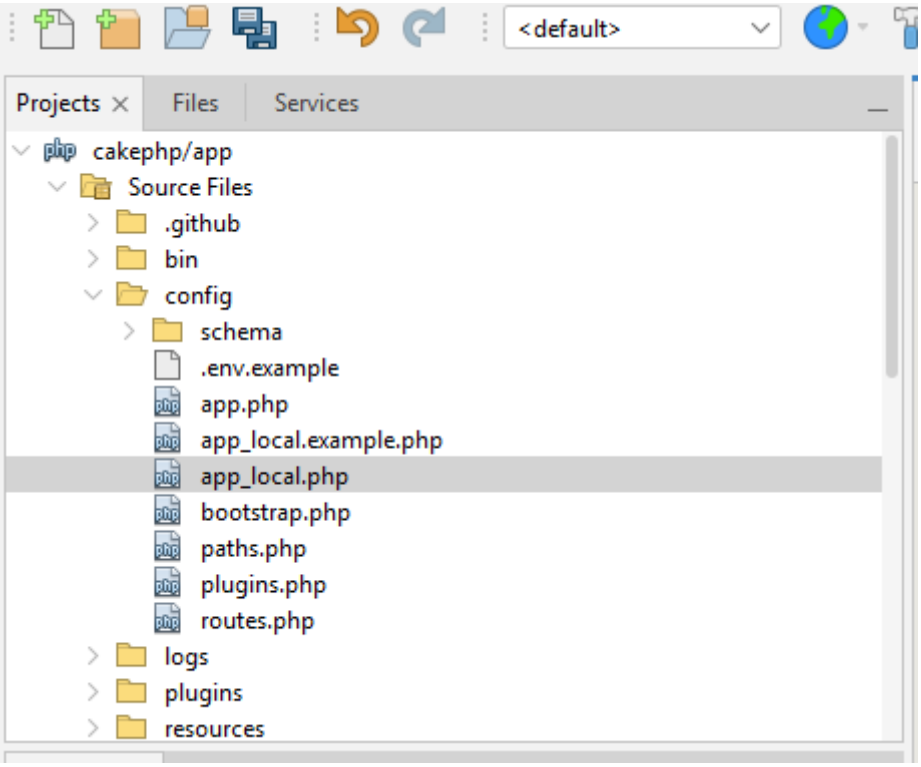


II – Migration

II.1 – Création migration pour la table article

```
mysql> create database blog_app_bdd
    -> ;
Query OK, 1 row affected (1.02 sec)
```

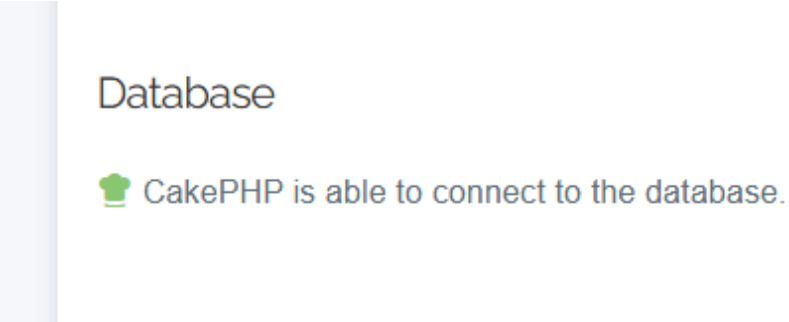
```
cake migrations create InitialMigration
```



- Ouvrir InitialMigration.php et ajouter dans la fonction change() :

```
*/
'Datasources' => [
    'default' => [
        'host' => 'localhost',
        /*
         * CakePHP will use the default DB port based on the driver selected
         * MySQL on MAMP uses port 8889, MAMP users will want to uncomment
         * the following line and set the port accordingly
         */
        //'port' => 'non_standard_port_number',

        'username' => 'blog_app',
        'password' => 'Xddll4*',

        'database' => 'blog_app_bdd',
```

```
public function change(): void
{
    $articles = $this->table('articles');
    $articles->create();
}
```




```
cake migrations migrate
```

```
C:\wamp64\www\blog_app\bin>cake migrations create InitialMigration

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001044707_InitialMigration.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001044707_InitialMigration.php`
```

- Vérifier dans phpMyAdmin la table créée :

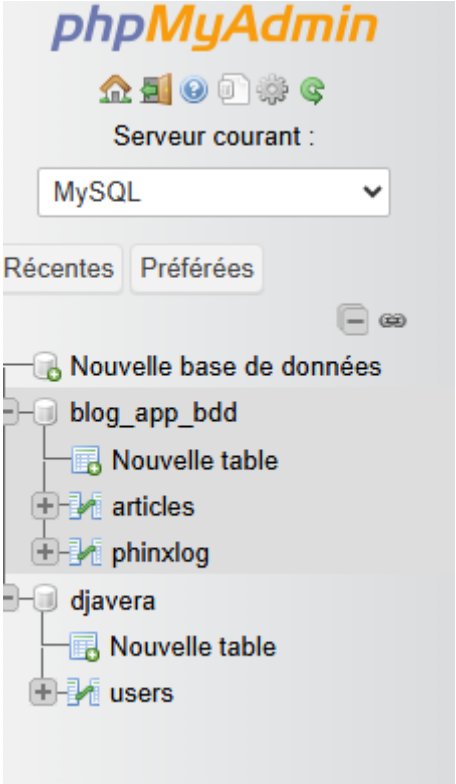
```php
class InitialMigration extends BaseMigration
{
    /**
     * Change Method.
     *
     * More information on this method is available here:
     * https://book.cakephp.org/migrations/4/en/migrations.html#the-change-method
     *
     * @return void
     */
    public function change(): void
    {
        $articles = $this->table('articles');
        $articles->create();

    }
}
```

- Pour annuler, effectuer un rollback :

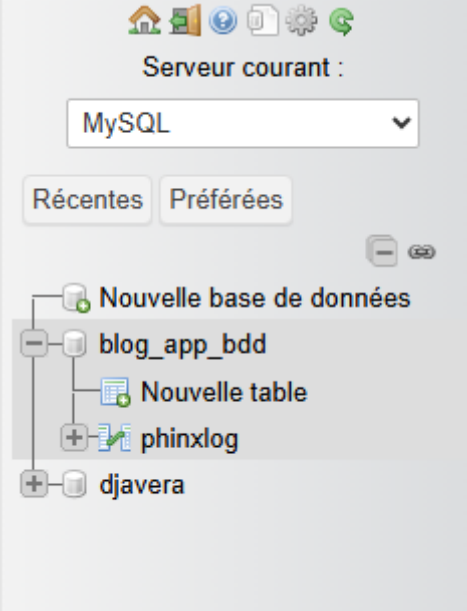
```
C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251001044707 InitialMigration: migrating
 == 20251001044707 InitialMigration: migrated 0.0586s
All Done. Took 0.3348s
```

```
C:\wamp64\www\blog_app\bin>cake migrations status
+--------+----------------+------------------+
| Status | Migration ID   | Migration Name   |
+--------+----------------+------------------+
| up     | 20251001044707 | InitialMigration |
+--------+----------------+------------------+
```

- Créer le fichier AlterArticles :




```
C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251001044707 InitialMigration: migrating
 == 20251001044707 InitialMigration: migrated 0.2679s
```



II.2 – Migration pour modifier la table articles

```
cake migrations create AlterArticles
```

```
C:\wamp64\www\blog_app\bin>cake migrations create AlterArticles

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001055315_AlterArticles.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001055315_AlterArticles.php`
```

```
$articles = $this->table('articles');
```

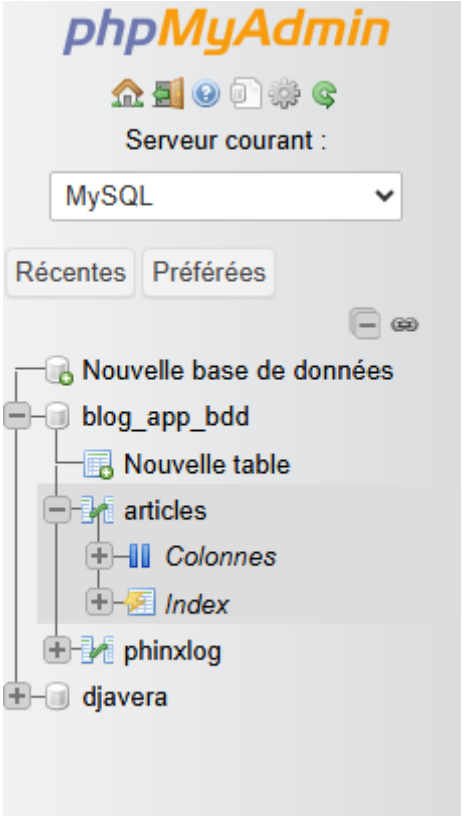
```
$articles->addColumn('title', 'string')
  ->addColumn('content', 'text')
  ->addColumn('created', 'datetime')
  ->addColumn('modified', 'datetime')
  ->update();
```

```
<?php
declare(strict_types=1);

use Migrations\BaseMigration;

class AlterArticles extends BaseMigration
{
    /**
     * Change Method.
     *
     * More information on this method is available here:
     * https://book.cakephp.org/migrations/4/en/migrations.html#the-change-metho
     *
     * @return void
     */
    public function change(): void
    {
        $articles = $this->table('articles');
$articles->addColumn('title', 'string')
->addColumn('content', 'text')
->addColumn('created', 'datetime')
->addColumn('modified', 'datetime')
->update();

    }
}
```

```
:\wamp64\www\blog_app\bin>cake migrations migrate

== 20251001060940 InsertArticles: migrating
== 20251001060940 InsertArticles: migrated 0.0694s
ll Done. Took 0.9913s
```

II.3 – Migration pour insérer des données dans articles

```
cake migrations create InsertArticles
```



```
C:\wamp64\www\blog_app\bin>cake migrations create InsertArticles

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001060940_InsertArticles.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001060940_InsertArticles.php`
```

III – Gestion des Articles

- Créer le modèle ArticlesTable dans src/Model/Table :

```php
    */
    public function up(): void
{
        $articles = $this->table('articles');
        $rows =[
            ['title' => 'titre1', 'content' => 'sfdsf'
                . 'dfsdfsd'
                . 'sdfsffs', 'created' => date('Y-m-d H:i:s'),  'modified' => date('Y-m-d H:i:s')],
            ['title' => 'titre2', 'content' => 'content2', 'created' => date('Y-m-d H:i:s'),  'modified' => date('Y-m-d
            ['title' => 'titre3', 'content' => 'content3', 'created' => date('Y-m-d H:i:s'),  'modified' => date('Y-m-d
            ['title' => 'titre4', 'content' => 'content4', 'created' => date('Y-m-d H:i:s'),  'modified' => date('Y-m-d
            ['title' => 'titre5', 'content' => 'content5', 'created' => date('Y-m-d H:i:s'),  'modified' => date('Y-m-d
        ];
        $articles->insert($rows)->saveData();
    }

    public function down(): void
    {
        $this->execute('DELETE from articles');
    }
}
```

```
+--------+----------------+-----------------------+
| Status | Migration ID   | Migration Name        |
+--------+----------------+-----------------------+
| up     | 20251001044707 | InitialMigration      |
| up     | 20251001055315 | AlterArticles         |
| up     | 20251001060940 | InsertArticles        |
```

```php
<?php
namespace App\Model\Table;
use Cake\ORM\Table;
class ArticlesTable extends Table {
    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
    }
}
```

Création du controller ArticlesController

Serveur : MySQL:3306 » Base de données : blog_app_bdd » Table : articles

Parcourir | Structure | SQL | Rechercher | Insérer | Exporter | Importer | Privilèges | Opérations | Déclencheurs

Affichage des lignes 0 - 4 (total de 5, traitement en 0,0007 seconde(s).)

```
SELECT * FROM `articles`
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

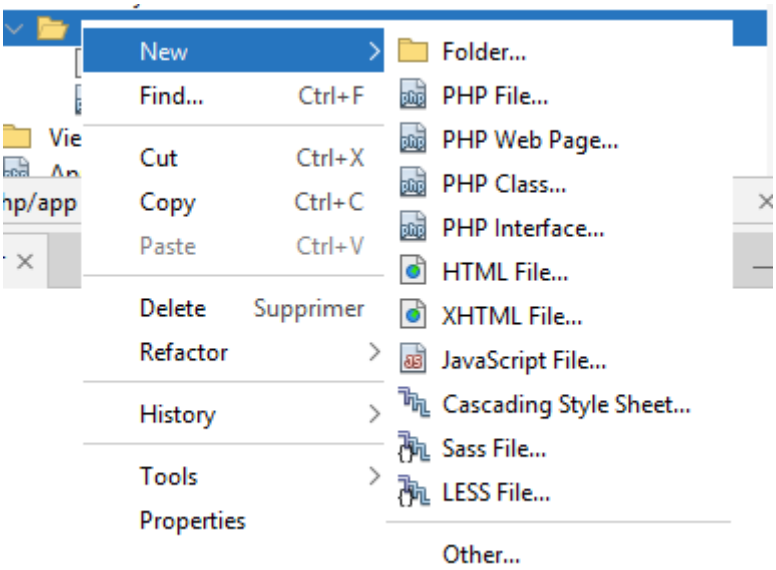
	id	title	content	created	modified
Éditer Copier Supprimer	1	titre1	sfdsfdfsdfsdsdfsffs	2025-10-01 06:45:34	2025-10-01 06:45:34
Éditer Copier Supprimer	2	titre2	content2	2025-10-01 06:45:34	2025-10-01 06:45:34
Éditer Copier Supprimer	3	titre3	content3	2025-10-01 06:45:34	2025-10-01 06:45:34
Éditer Copier Supprimer	4	titre4	content4	2025-10-01 06:45:34	2025-10-01 06:45:34
Éditer Copier Supprimer	5	titre5	content5	2025-10-01 06:45:34	2025-10-01 06:45:34

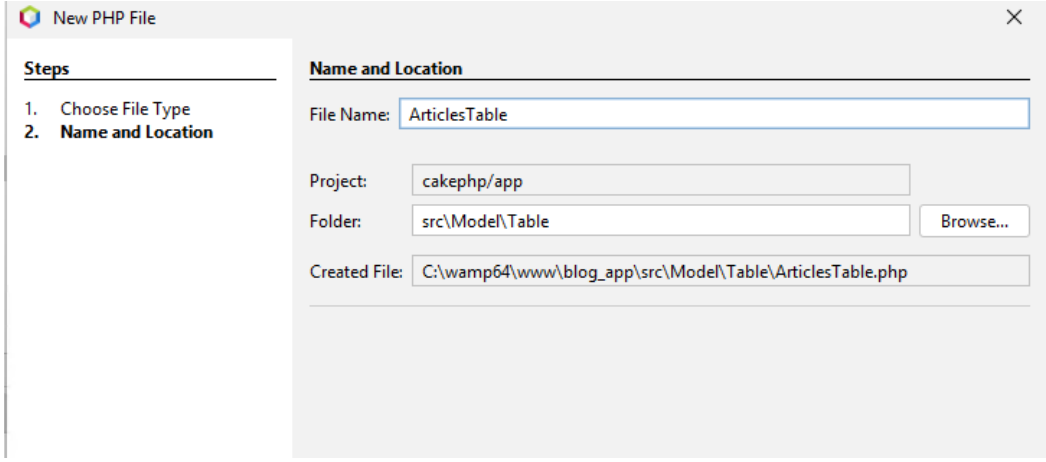
Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

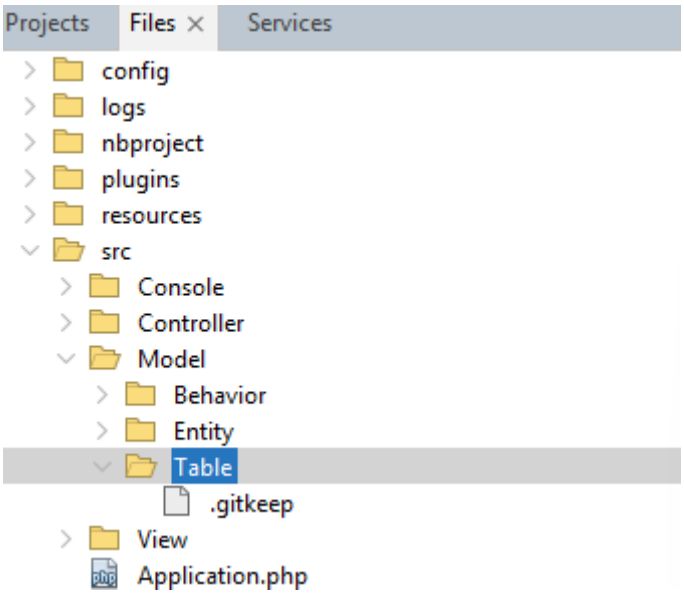
```php
<?php
namespace App\Controller;
class ArticlesController extends AppController {
    public function index() {
        $mesArticles = $this->Articles->find()->all();
        $this->set(compact('mesArticles'));
    }
}
```

III.1 – Création de l'index pour Articles

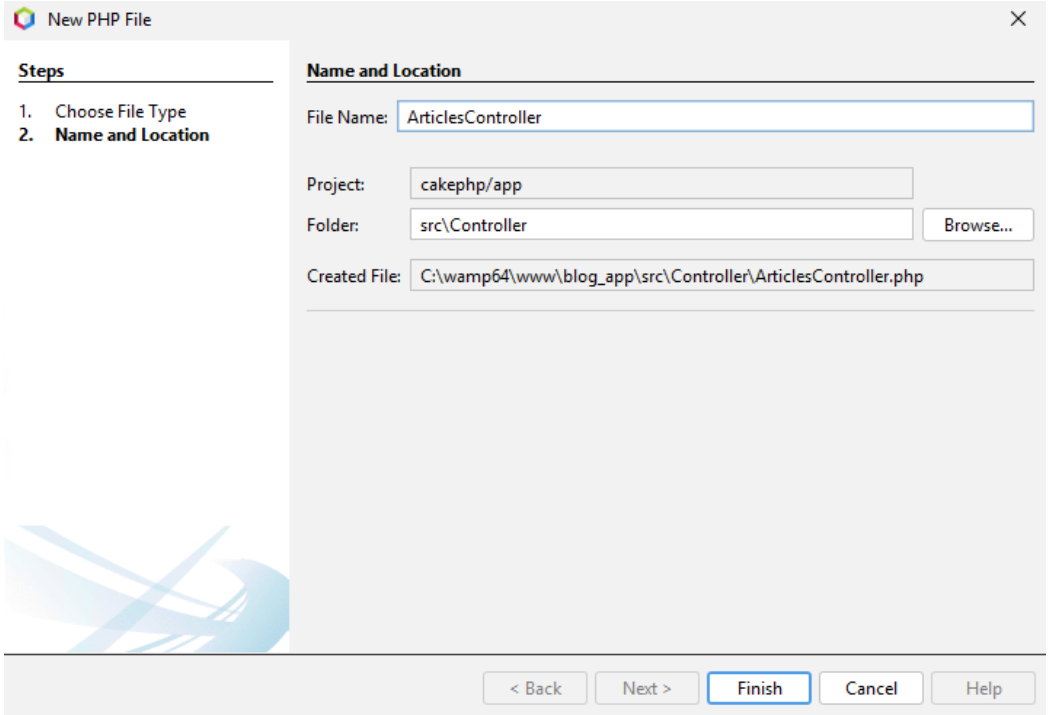
- Créer le dossier Articles dans src/View et le fichier index.php :





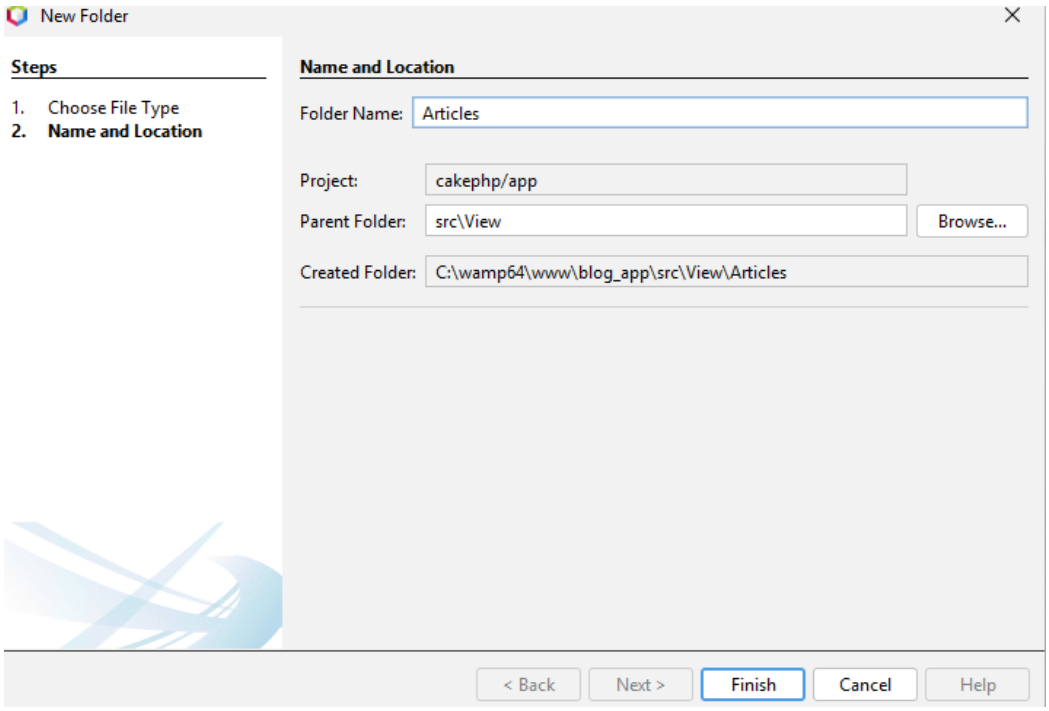


```php
<?php

namespace App\Model\Table;

use Cake\ORM\Table;

class ArticlesTable extends Table {

}
```

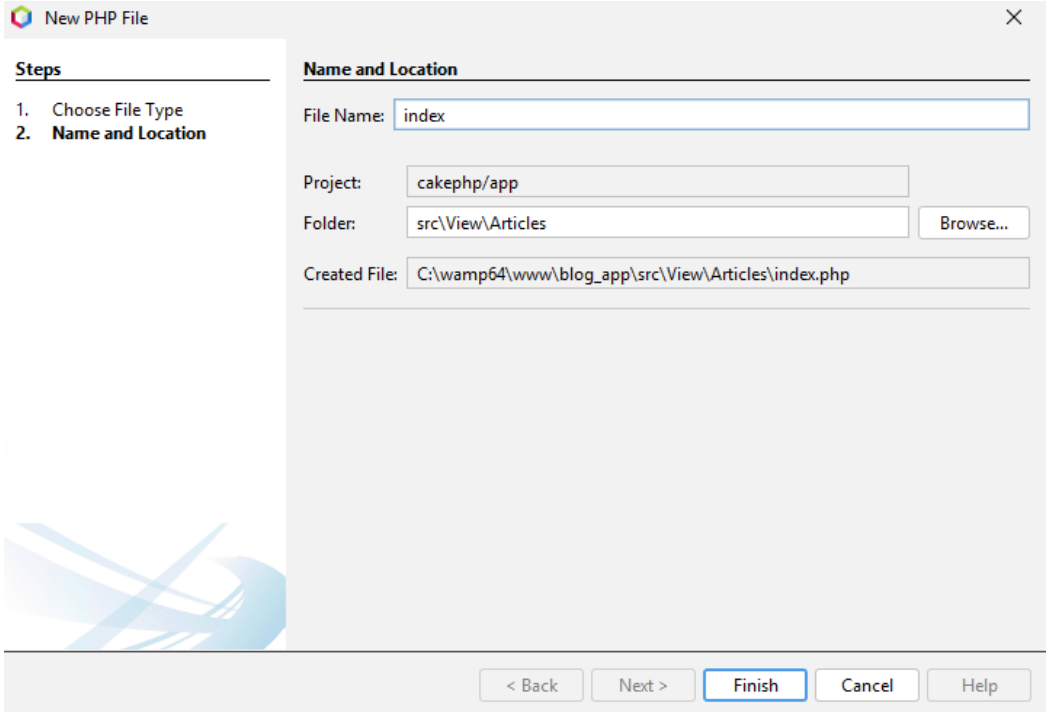


Résultat sur le site

```php
<?php

namespace App\Controller;

class ArticlesController extends AppController {

    public function index() {
        //on récupére tous les posts et on les stocke dans $mesArticles
        $mesArticles = $this->Articles->find()->all();
        $this->set(compact('mesArticles'));
    }

}
```



- Modifier le code pour afficher la date de création et de modification :



Source History

```
<h1>Tous les articles du Blog</h1>
<table>
<tr>
        <th>Id</th>
        <th>Titre</th>
        <th>Date de cr&eacute;ation</th>
    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'afficha
    <?php foreach ($mesArticles as $article): ?>
    <tr>
        <td><?= $article->id ?></td>
        <td><?= $article->title ?></td>
        <td><?= $article->created->format(DATE_RFC850) ?></td>
    </tr>
    <?php endforeach; ?>
</table>
```

localhost/blog_app/articles

CakePHP

Tous les articles du Blog

Id	Titre	Date de création
1	titre1	Wednesday, 01-Oct-25 06:45:34 UTC
2	titre2	Wednesday, 01-Oct-25 06:45:34 UTC
3	titre3	Wednesday, 01-Oct-25 06:45:34 UTC
4	titre4	Wednesday, 01-Oct-25 06:45:34 UTC
5	titre5	Wednesday, 01-Oct-25 06:45:34 UTC

```
<tr>
    <th>Id</th>
    <th>Titre</th>
    <th>Nb commentaires</th>
    <th>Date de cr&eacute;ation</th>
    <th>Date de modification</th>

    <th>Action<th>
</tr>

<!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l
<?php foreach ($mesArticles as $article): ?>
    <tr>
        <td><?= $article->id ?></td>

        <td><?= $article->created->format(DATE_RFC850) ?></td>
        <td><?= $article->modified->format(DATE_RFC850) ?></td>
```

Source History

```
<h1>Tous les articles du Blog</h1>
<table>
    <tr>
        <th>Id</th>
        <th>Titre</th>
        <th>Date de cr&eacute;ation</th>
        <th>Date de modified</th>


    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'
    <?php foreach ($mesArticles as $article): ?>
        <tr>
            <td><?= $article->id ?></td>
            <td>
                <?=
                $this->html->link($article->title, [
                    'controller' => 'articles',
                    'action' => 'detail',
                    $article->id]);
//l'url généré sera de la forme /articles/detail/…
                ?>
            </td>
            <td><?= $article->created->format(DATE_RFC850) ?></td>
            <td><?= $article->modified->format(DATE_RFC850) ?></td>

        </tr>
<?php endforeach; ?>
</table>
```

localhost/blog_app/articles

CakePHP Documentation API

Tous les articles du Blog

Id	Titre	Date de création	Date de modified
1	titre1	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
2	titre2	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
3	titre3	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
4	titre4	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
5	titre5	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC

Source History

```php
<?php

namespace App\Controller;

class ArticlesController extends AppController {

    public function index() {
        //on récupére tous les posts et on les stocke dans $mesArticles
        $mesArticles = $this->Articles->find()->all();
        $this->set(compact('mesArticles'));
        }


        public function detail ($id){

        $leArticle = $this->Articles->get($id);
        $this->set(compact('leArticle'));
    }
}
```

localhost/blog_app/articles

CakePHP Documentation API

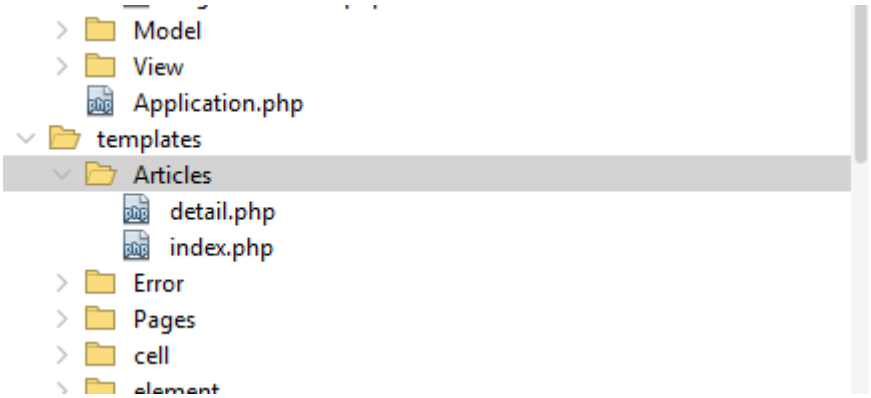
Tous les articles du Blog

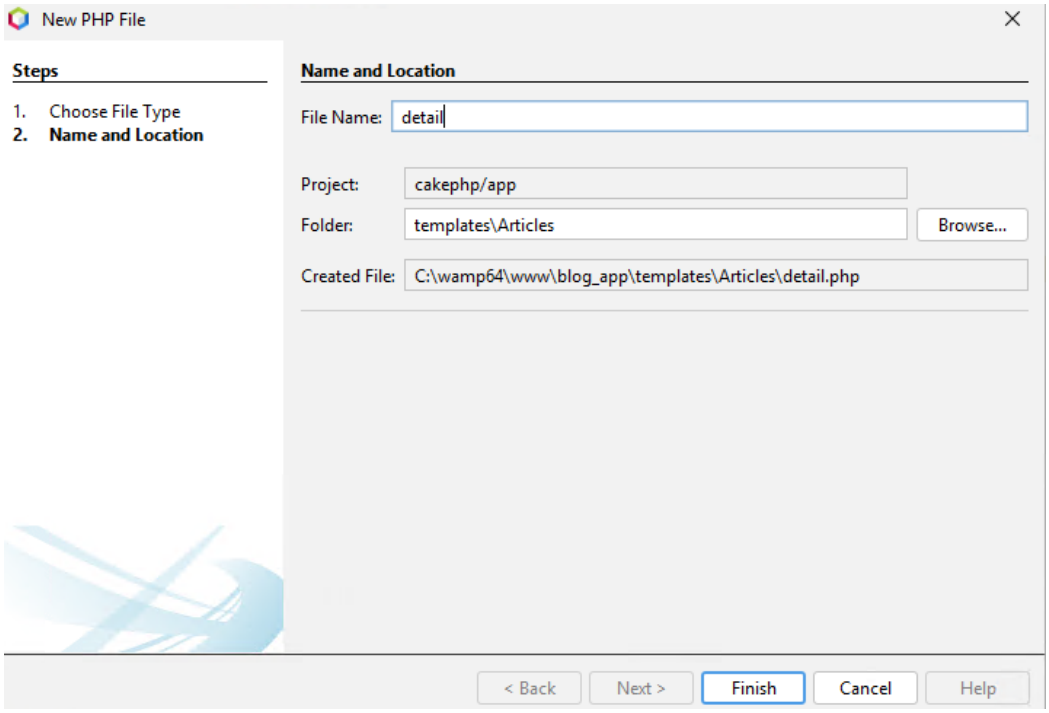
Id	Titre	Date de création	Date de modified
1	titre1	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
2	titre2	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
3	titre3	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
4	titre4	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
5	titre5	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC

III.2 – Créez la vue pour detail

```
Source  History
<h1>Tous les articles du Blog</h1>
<table>
    <tr>
        <th>Id</th>
        <th>Titre</th>
        <th>Date de cr&eacute;ation</th>
        <th>Date de modified</th>


    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'
    <?php foreach ($mesArticles as $article): ?>
        <tr>
            <td><?= $article->id ?></td>
            <td>
                <?=
                $this->html->link($article->title, [
                    'controller' => 'articles',
                    'action' => 'detail',
                    $article->id]);
//l'url généré sera de la forme /articles/detail/…
                ?>
            </td>
            <td><?= $article->created->format(DATE_RFC850) ?></td>
            <td><?= $article->modified->format(DATE_RFC850) ?></td>

        </tr>
<?php endforeach; ?>
</table>
```

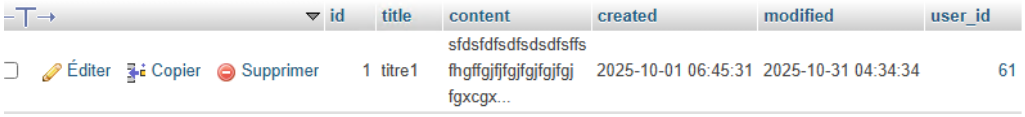
- Créer le fichier templates/Articles/detail.php :



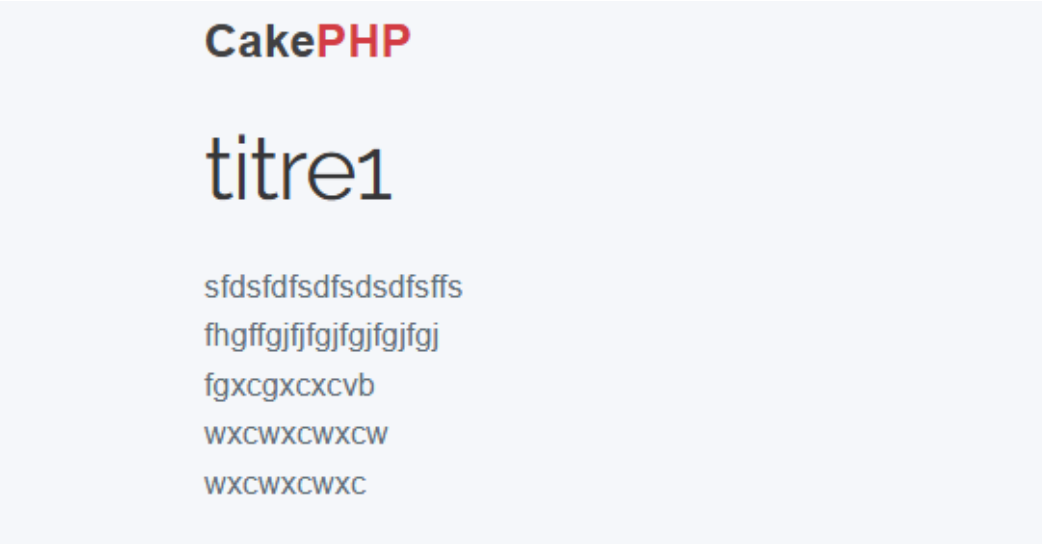



```
<h1><?= h($leArticle->title) ?></h1>
<p><?= nl2br($leArticle->content) ?></p>
<p>
    <small>Created : <?= $leArticle->created->format(DATE_RFC850) ?></small><br/>
    <small>Created by : <?= $leArticle->user->username ?></small><br/>

</p>
```

```
<h1><?= h($leArticle->title) ?></h1>
<p><?= nl2br($leArticle->content) ?></p>
<p>
  <small>Created : <?= $leArticle->created->format(DATE_RFC850) ?></small><br/>
  <small>Created by : <?= $leArticle->user->username ?></small><br/>
</p>
<?= $this->Html->link('Retour à la liste des articles',
  ['controller' => 'articles', 'action' => 'index'],
  ['class' => 'button']); ?>
```



	id	title	content	created	modified	user_id
Éditer Copier Supprimer	1	titre1	sfdsfdfsdfsdsdfsffs fhgffgjfjfgjfgjfgjfgj fgxcgx...	2025-10-01 06:45:31	2025-10-31 04:34:34	61



```php
<?=
$this->html->link('Retour à la liste des articles', ['controller' => 'articles',
    'action' => 'index'],['class' => 'button']);
?>
```

III.3 – Fonction add

```php
<?php

namespace App\Controller;

class ArticlesController extends AppController {

    public function index() {
        //on récupère tous les posts et on les stocke dans $mesArticles
        $mesArticles = $this->Articles->find()->all();
        $this->set(compact('mesArticles'));
    }

public function detail($id = null) {
        try {
            $leArticle = $this->Articles->get($id);
        } catch (\Exception $ex) {
            if ($id == null) {
                $this->Flash->error(__("L'action detail doit être appelé avec un identifiant"));
            } else {
                $this->Flash->error(__("L'article {0} n'existe pas", $id));
            }
            return $this->redirect(['action' => 'index']);
        }

        $this->set(compact('leArticle'));
    }
}
```

```php
public function add() {
    $leNewArticle = $this->Articles->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewArticle = $this->Articles->patchEntity(
            $leNewArticle, $this->request->getData());
        if ($this->Articles->save($leNewArticle)) {
            $this->Flash->success(__("L'articles a été sauvegardé."));
            return $this->redirect(['action' => 'index']);
```

```
        } else {
            $this->Flash->error(__("Impossible d'ajouter votre article."));
        }
    }
    $this->set(compact('leNewArticle'));
}
```

CakePHP Documentation API

L'action detail doit être appelé avec un identifiant

Tous les articles du Blog

Id	Titre	Date de création	Date de modified
1	titre1	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
2	titre2	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
3	titre3	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
4	titre4	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC
5	titre5	Wednesday, 01-Oct-25 06:45:34 UTC	Wednesday, 01-Oct-25 06:45:34 UTC

L'article 210 n'existe pas Record not found in table `articles`.

Tous les articles du Blog

III.4 – Créer une vue pour add

localhost/blog_app/articles/detail/1

CakePHP

titre1

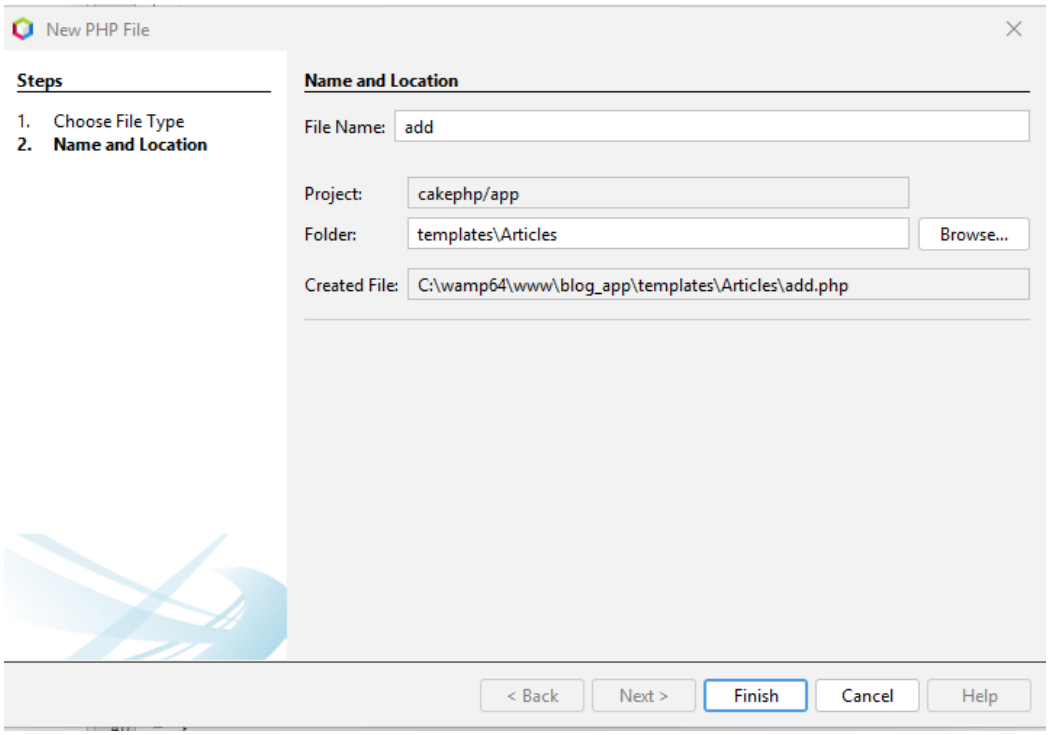
sfdsfdfsdfsdsdfsffs
fhgffgjfjfgjfgjfgjfgj
fgxcgxcxcvb
wxcwxcwxcw
wxcwxcwxc

Created : Wednesday, 01-Oct-25 06:45:31 UTC

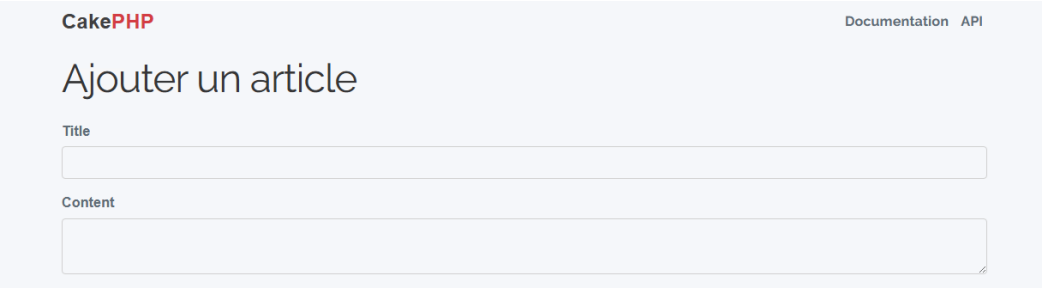
Retour à la liste des articles

```php
public function add() {
    $leNewArticle = $this->Articles->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewArticle = $this->Articles->patchEntity($leNewArticle, $this->request->data);
        if ($this->Articles->save($leNewArticle)) {
            $this->Flash->success(__("L'articles a été sauvegardé."));
            return $this->redirect(['action' => 'index']);
        } else
            $this->Flash->error(__("Impossible d'ajouter votre article."));
    }
    $this->set(compact('leNewArticle'));
}
```

```php
<h1>Ajouter un article</h1>
<?php
    echo $this->Form->create($leNewArticle);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Sauvegarder l'article"));
    echo $this->Form->end();
?>
<br/>
<?= $this->Html->link('Retour à la liste des articles',
    ['controller' => 'articles', 'action' => 'index'],
    ['class' => 'button']); ?>
```



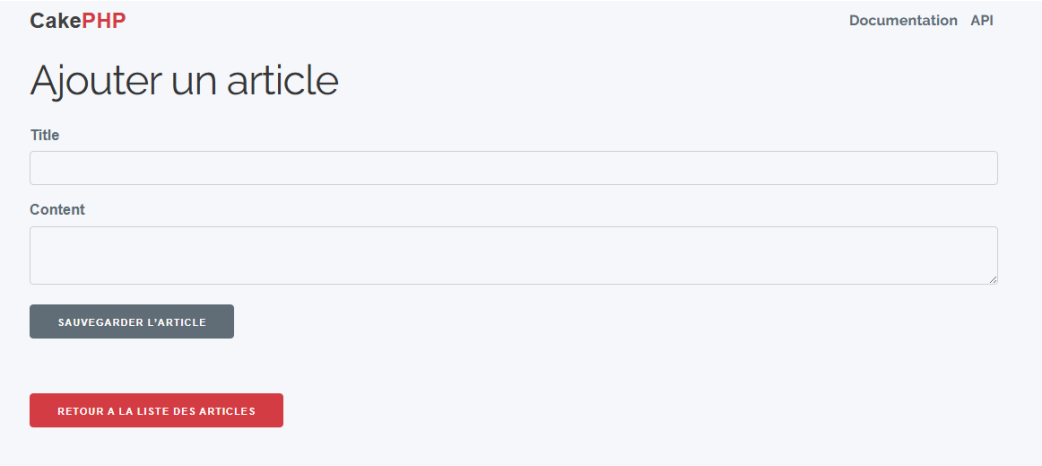

```php
<h1>Ajouter un article</h1>
<?php
echo $this->Form->create($leNewArticle);
echo $this->Form->control('title');
echo $this->Form->control('content', ['rows' => '3']);
echo $this->Form->button(__("Sauvegarder l'article"));
echo $this->Form->end();
?>
<br/>
<?php unset($lePost); ?>
```



III.5 – Validateur

- Dans ArticlesTable.php, ajouter la fonction validationDefault :

```
<?=
$this->html->link('Retour a la liste des articles', [
    'controller' => 'articles', 'action' => 'index'],
        ['class' => 'button']);
?>
```




```
use Cake\Validation\Validator;

public function validationDefault(Validator $validator): Validator {
    $validator
        ->notEmptyString('title', __("veuillez renseigner un title"))
        ->notEmptyString('content', __("veuillez renseigner un content"));
    return $validator;
}
```

```
<h1>Tous les articles du Blog</h1>
<?=
$this->html->link('Ajouter un article', ['controller' => 'articles',
    'action' => 'add'], ['class' => 'button']);
?>
```

III.6 – Editer un article

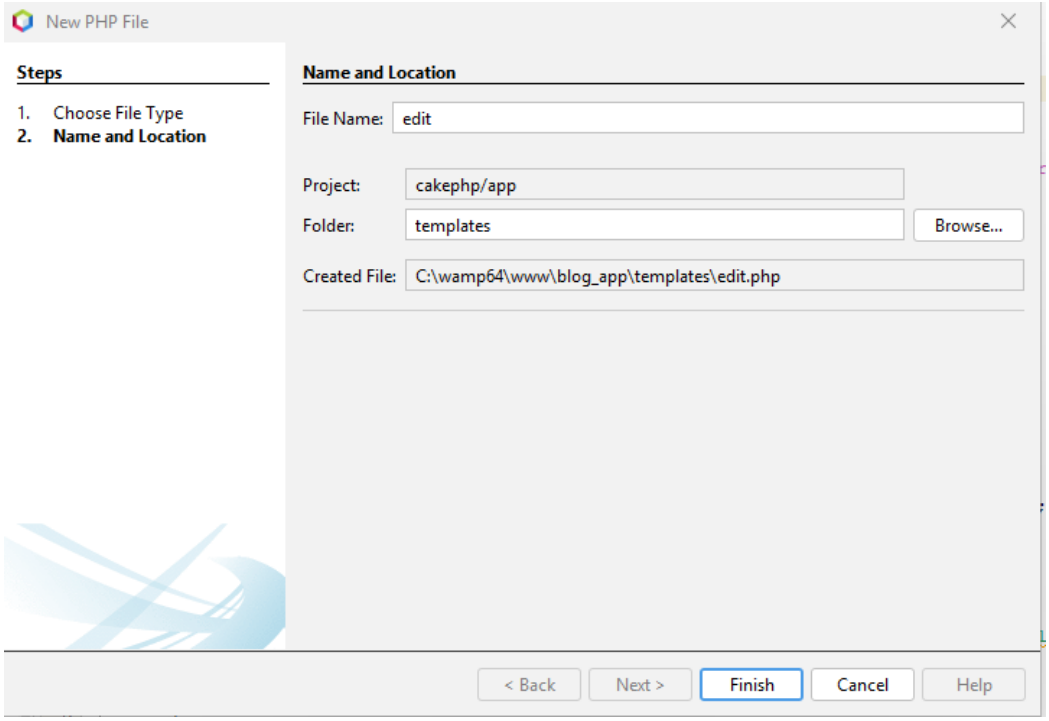
- Ajouter la fonction edit dans ArticlesController :

```php
public function edit($id) {
    $leArticle = $this->Articles->get($id);
    if ($this->request->is(['post', 'put'])) {
        $this->Articles->patchEntity($leArticle, $this->request->getData());
        if ($this->Articles->save($leArticle)) {
            $this->Flash->success(__("Votre article a été mis à jour."));
            return $this->redirect(['action' => 'index']);
        } else {
            $this->Flash->error(__("Impossible de mettre à jour votre article."));
        }
    }
    $this->set(compact('leArticle'));
}
```

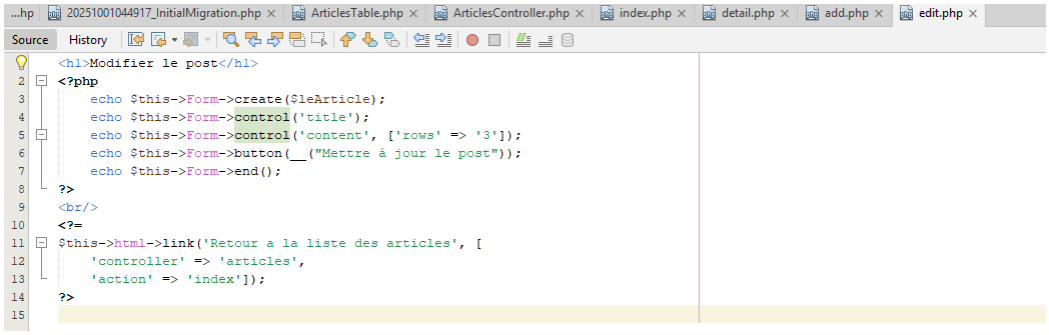
```php
<?php

namespace App\Model\Table;

use Cake\ORM\Table;
use cake\Validation\Validator;

class ArticlesTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('title', __('veuillez renseigner un title'))
                ->notEmptyString('content', __('veuillez renseigner un content'));

        return $validator;
    }
}
```

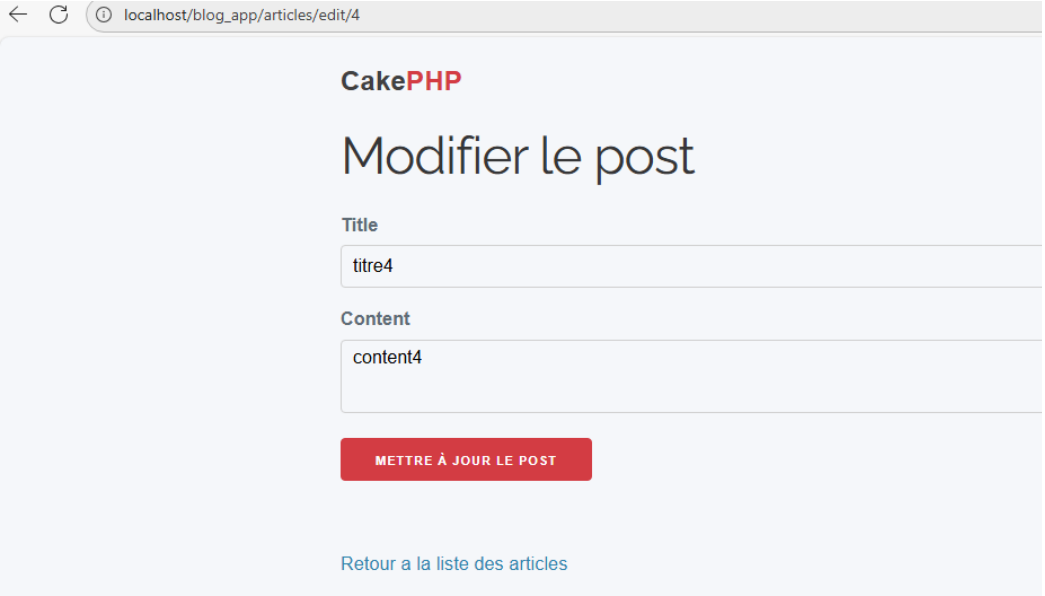
III.7 – Créez vue edit

```php
    public function edit($id) {
        $leArticle = $this->Articles->get($id);
        if ($this->request->is(['post', 'put'])) {
            $this->Articles->patchEntity($leArticle, $this->request->getData());
            if ($this->Articles->save($leArticle)) {
                $this->Flash->success(__('Votre article a été mis à jour.'));
                return $this->redirect(['action' => 'index']);
            } else
                $this->Flash->error(__('Impossible de mettre à jour votre article.'));
        }
        $this->set(compact('leArticle'));
    }
}
```




```php
<h1>Modifier le post "<?= $leArticle->title?>" (id - <?= $leArticle->id ?>)</h1>
<?php
    echo $this->Form->create($leArticle);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Mettre à jour le post"));
    echo $this->Form->end();
?>
<?= $this->Html->link('Retour a la liste des articles',
    ['controller' => 'articles', 'action' => 'index']); ?>
```





- Ajouter bouton supprimer en couleur et changer $post par $article :

```
// Bouton Edit dans l'index
<?= $this->Html->link('Button',
    ['controller' => 'articles', 'action' => 'edit', $article->id],
    ['class' => 'button']); ?>

// Bouton Supprimer avec confirmation
<?= $this->Form->postLink(__("Supprimer"),
    ['action' => 'delete', $article->id],
    ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1}",
        $article->title, $article->id),
     'class' => 'button', 'style' => 'background-color: blue']); ?>

// Fonction delete dans le controller
public function delete($id = null) {
    $this->request->allowMethod(['post', 'delete']);
    $leArticle = $this->Articles->get($id);
    if ($this->Articles->delete($leArticle)) {
        $this->Flash->success(__("L'article {0} d'id {1} a bien été supprimé !",
            $leArticle->title, $leArticle->id));
    }
    return $this->redirect(['action' => 'index']);
}
```

```
<td>
    <?=
    $this->html->link('Button', [
        'controller' => 'articles', 'action' => 'edit'],
            ['class' => 'button']);
    ?>
</td>
```

Id	Titre	Date de création	Date de modification	Créer par	Action
1	titre1	Wednesday, 01-Oct-25 06:45:31 UTC	Friday, 31-Oct-25 04:34:34 UTC	test1	EDIT
2	titre2	Wednesday, 01-Oct-25 06:45:31 UTC	Friday, 31-Oct-25 04:34:42 UTC	test2	EDIT
3	titre3	Wednesday, 01-Oct-25 06:45:31 UTC	Wednesday, 01-Oct-25 06:45:31 UTC	test1	EDIT
4	titre4	Wednesday, 01-Oct-25 06:45:31 UTC	Wednesday, 01-Oct-25 06:45:31 UTC	test1	EDIT
5	titre5	Wednesday, 01-Oct-25 06:45:31 UTC	Wednesday, 01-Oct-25 06:45:31 UTC	test1	EDIT

```
<?=
$this->html->link('Retour à la liste des articles', ['controller' => 'articles',
    'action' => 'index'], ['class' => 'button', 'style'=> 'background-color : blue']);
?>
```

IV – Gestion des Users

IV.1 – Création de la table User

```
public function delete($id = null) {
    $this->request->allowMethod(['post', 'delete']);
    $leArticle = $this->Articles->get($id);
    if ($this->Articles->delete($leArticle)) {
        $this->Flash->success(__("L'article {0} d' id {1} a bien été supprimé ! ", $leArticle->title, $leArticle->id));
    } else {
        $this->Flash->error(__("L'article ne peux pas etre supprimé, reessayez plus tard"));
    }
    return $this->redirect(['action' => 'index']);
}
```

```
cake bake migration CreateUsers name:string lastname:string age:integer email:string
username:string password:string created modified
```

```
<?=
$this->Form->postLink(
        __('Supprimer'),
        ['action' => 'delete', $post->id],
        ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $post->title, $post->id)])
?>
```

```
<?=
$this->Form->postLink(
        __('Supprimer'),
        ['action' => 'delete', $article->id],
        ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $article->title, $article->id)])
?>
```

```
<td>
    <?=
    $this->Form->postLink(
            __('Supprimer'),
            ['action' => 'delete', $article->id],
            ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $article->title, $article->id),
                'class' => 'button', 'style' => 'background-color: blue']);
    ?>
</td>
```

```
C:\wamp64\www\blog_app\bin>cake bake migration CreateUsers name:string lastname:string age:integer email:string username
:string password:string created modified
```

IV.2 – Création des templates de user

- Dans UsersController.php :

```
C:\wamp64\www\blog_app\bin>cake migrations create InsertUsers
```

```
class UsersController extends AppController {
    public function index() {
        $mesUsers = $this->Users->find()->all();
        $this->set(compact('mesUsers'));
    }
}
```

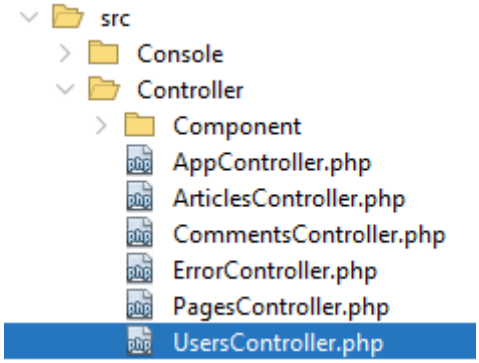
- Dans UsersTable.php :

```
public function up(): void {
    $users = $this->table('users');
    $rows = [
        ['name' => 'Bob', 'lastname' => 'Lenon', 'age' => 18, 'email' => "fsdfsd1@gmail.com", 'username' => 'test1', 'password' => 'dsfdsfd
        ['name' => 'Fred', 'lastname' => 'Dupont', 'age' => 20, 'email' => "fsdfsd2@gmail.com", 'username' => 'test2', 'password' => 'sfdsf
        ['name' => 'Paul', 'lastname' => 'Dubard', 'age' => 33, 'email' => "fsdfsd3@gmail.com", 'username' => 'test3', 'password' => 'sfdsf
        ['name' => 'Ted', 'lastname' => 'Lefeve', 'age' => 35, 'email' => "fsdfsd4@gmail.com", 'username' => 'test4', 'password' => 'sfdsfd
        ['name' => 'Jean', 'lastname' => 'Maurice', 'age' => 40, 'email' => "fsdfsd5@gmail.com", 'username' => 'test5', 'password' => 'sfds
    ];
```

```
class UsersTable extends Table {
    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->hasMany('Articles');
    }
    public function validationDefault(Validator $validator): Validator {
        $validator
            ->add('username', 'unique', ['rule' => 'validateUnique',
                'provider' => 'table', 'message' => 'Ce username est déjà utilisé'])
            ->notEmptyString('password', 'Le mot de passe est requis')
```

```
            ->notEmptyString('name', 'Vous devez fournir un prénom')
            ->notEmptyString('lastname', 'Vous devez fournir un nom')
            ->notEmptyString('age', 'Vous devez fournir un age')
            ->naturalNumber('age', "L'âge doit être un nombre")
            ->email('email', false, "Le format de l'email est invalide")
            ->add('email', 'unique', ['rule' => 'validateUnique',
                'provider' => 'table', 'message' => 'Cet email est déjà utilisé']);
        return $validator;
    }
}
```

- Dans Index user :




```
<?php

namespace App\Controller;

class UsersController extends AppController {

    public function index() {
        //on récupère tous les utilisateurs et on les stocke dans $mesUsers
        $mesUsers = $this->Users->find()->all();
        $this->set(compact('mesUsers')); //envoie à la vue le contenu de $mesUsers dans $rep qui sera utiliseable
    }
}
```

```
<?php

namespace App\Model\Table;

use Cake\ORM\Table;
use Cake\Validation\Validator;

class UsersTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('title', __('Veuillez renseigner un titre'))
                ->notEmptyString('content', _('Veuillez renseigner une description'));

        return $validator;
    }
}
```

```php
<table>
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Lastname</th>
        <th>Age</th>
        <th>Email</th>
        <th>Username</th>
        <td>nombre d'articles</td>
        <th>date de création</th>
        <th>date de modification</th>
        <th>action</th>

    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'affichage des infos des articles -->
    <?php foreach ($mesUsers as $user): ?>
        <tr>
            <td><?= $user->id ?></td>
            <td><?= $user->name ?></td>
            <td><?= $user->lastname ?></td>
            <td><?= $user->age ?></td>
            <td><?= $user->email ?></td>
            <td><?= $user->username ?></td>
            <td><?= count($user->articles) ?></td>
            <td><?= $user->created->format(DATE_RFC850) ?></td>
            <td><?= $user->modified->format(DATE_RFC850) ?></td>
            <td>
                <div style="display:flex; align-items:center; gap:20px;">
    <?= $this->Html->link(
        $this->Html->image('edit.png', ['alt' => 'edit']),
        ['controller' => 'Users', 'action' => 'edit', $user->id],
        ['escape' => false]
    ) ?>

    <?= $this->Form->postLink(
        $this->Html->image('delete.png', ['alt' => 'delete']),
        ['action' => 'delete', $user->id],
```

Activer Windows

```php
<?php

namespace App\Model\Table;

use Cake\ORM\Table;
use Cake\Validation\Validator;

class UsersTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->hasMany('Articles');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->add('username', 'unique', [
                    'rule' => 'validateUnique',
                    'provider' => 'table',
                    'message' => 'Ce username est déjà utilisé'])
                ->notEmptyString('password', 'Le mot de passe est requis')
                ->notEmptyString('name', 'Vous devez fournir un prénom')
                ->notEmptyString('lastname', 'Vous devez fournir un nom')
                ->notEmptyString('age', 'Vous devez fournir un age')
                ->naturalNumber('age', 'L\'âge doit être un nombre')
                ->email('email', false, 'Le format de l\'email est invalide')
                ->add('email', 'unique', [
                    'rule' => 'validateUnique',
                    'provider' => 'table',
                    'message' => 'Cet email est déjà utilisé'
        ]);

        return $validator;
    }
}
```

V – Changer la page d'accueil de CakePHP

- Dans config/routes.php, modifier la route par défaut :

```
*/
//$builder->connect('/', ['controller' => 'Pages', 'action' => 'display', 'home']);
$builder->connect('/', ['controller' => 'Articles', 'action' => 'index']);
```

```
// Commenter l'ancienne route :
//$builder->connect('/', ['controller' => 'Pages', 'action' => 'display', 'home']);

// Ajouter la nouvelle :
$builder->connect('/', ['controller' => 'Articles', 'action' => 'index']);
```

Nouvel onglet | CakePHP: the rapid de | localhost / MySQL / bl | CakePHP: the rapid de

localhost/blog_app/

```
public function index() {
    //on récupère tous les posts et on les stocke dans $mesArticles
    $mesArticles = $this->Articles->find()->contain(['Users'])->all();
    $this->set(compact('mesArticles'));
}
```

```
public function index() {
    //on récupère tous les posts et on les stocke dans $mesArticles
    $mesArticles = $this->Articles->find()->contain([
            'Users' => function ($q) {
                return $q
                    ->select(['username', 'email']);
            }])->all();
    $this->set(compact('mesArticles'));
}
```

```
$article->id]);
'l'url généré sera de la forme /articles/detail/…
?>
</td>
<td><?= $article->created->format(DATE_RFC850) ?></td>
<td><?= $article->modified->format(DATE_RFC850) ?></td>
<td><?= $article->user->username ?></td>
<td>
```

VI – Manipulation et affichage des données

VI.1 – Récupération des données

- Modifier la fonction index dans ArticlesController pour récupérer les Users :

Id	Titre	Nb commentaires	Date de création	Date de modification	Créer par
1	titre1	1	Wednesday, 12-Nov-25 09:13:34 UTC	Wednesday, 12-Nov-25 09:13:34 UTC	test1
2	titre2	0	Wednesday, 12-Nov-25 09:13:34 UTC	Wednesday, 19-Nov-25 06:57:16 UTC	test2
3	titre3	0	Wednesday, 12-Nov-25 09:13:34 UTC	Wednesday, 19-Nov-25 06:57:21 UTC	test3
4	titre4	0	Wednesday, 12-Nov-25 09:13:34 UTC	Wednesday, 19-Nov-25 06:57:26 UTC	test4
5	titre5	0	Wednesday, 12-Nov-25 09:13:34 UTC	Wednesday, 12-Nov-25 09:13:34 UTC	test1

```
public function index() {
    $mesArticles = $this->Articles->find()->contain([
        'Users' => function ($q) {
            return $q->select(['username', 'email']);
        }
    ])->all();
    $this->set(compact('mesArticles'));
}
```

```
<tr>
    <th>Id</th>
    <th>Name</th>
    <th>LastName</th>
    <th>Age</th>
    <th>Username</th>
    <th>Email</th>
    <th>NB ARTICLES</th>
    <th>Date de cr&eacute;ation</th>
    <th>Date de modification</th>
    <th>Actions</th>
</tr>
```

VI.2 – Affichage du créateur d'un article

- Dans la vue index, ajouter les colonnes :

```
<!-- ici se trouve l'iteration sur l'objet query de notre $mesArticles, l'affichag
<?php foreach ($mesUsers as $user): ?>
    <tr>
        <td><?= $user->id ?></td>
        <td><?= $user->name ?></td>
        <td><?= $user->lastname ?></td>
        <td><?= $user->age ?></td>
        <td><?= $user->username ?></td>
        <td><?= $user->email ?></td>
        <td><?= count($user->articles) ?></td>
        <td><?= $user->created->format(DATE_RFC850) ?></td>
        <td><?= $user->modified->format(DATE_RFC850) ?></td>
        <td>
```

Id	Name	Lastname	Age	Email	Username	nombre d'articles	date de création	date de modification	action
61	Bob	Lenon	18	fsdfsd1@gmail.com	test1	0	Friday, 10-Oct-25 05:03:47 UTC	Wednesday, 29-Oct-25 03:45:27 UTC	EDIT SUPPRIMER
62	Fred	Dupont	20	fsdfsd2@gmail.com	test2	0	Friday, 10-Oct-25 05:03:47 UTC	Friday, 10-Oct-25 05:03:47 UTC	EDIT SUPPRIMER
63	Paul	Dubard	33	fsdfsd3@gmail.com	test3	1	Friday, 10-Oct-25 05:03:47 UTC	Friday, 10-Oct-25 05:03:47 UTC	EDIT SUPPRIMER

```
<td><?= $article->created->format(DATE_RFC850) ?></td>
<td><?= $article->modified->format(DATE_RFC850) ?></td>
<td><?= $article->user->username ?></td>
```

```
C:\wamp64\www\blog_app\bin>cake bake migration CreateComments title:string content:string created article_id:int

Creating file C:\wamp64\www\blog_app\config\Migrations\20251105042940_CreateComments.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251105042940_CreateComments.php`
```

VI.3 – Nombre d'articles par user

- Dans la vue index des users, ajouter la colonne NB ARTICLES :

```php
use Migrations\BaseMigration;

class CreateComments extends BaseMigration
{
    /**
     * Change Method.
     *
     * More information on this method is available here:
     * https://book.cakephp.org/migrations/4/en/migrations.html#the-change-method
     *
     * @return void
     */
    public function change(): void
    {
        $table = $this->table('comments');
        $table->addColumn('title', 'string', [
            'default' => null,
            'limit' => 255,
            'null' => false,
        ]);
        $table->addColumn('content', 'string', [
            'default' => null,
            'limit' => 255,
            'null' => false,
        ]);
        $table->addColumn('created', 'datetime', [
            'default' => null,
            'null' => false,
        ]);
        $table->addColumn('article_id', 'integer', [
            'null' => false,
        ]);
        $table->create();
    }
}
```

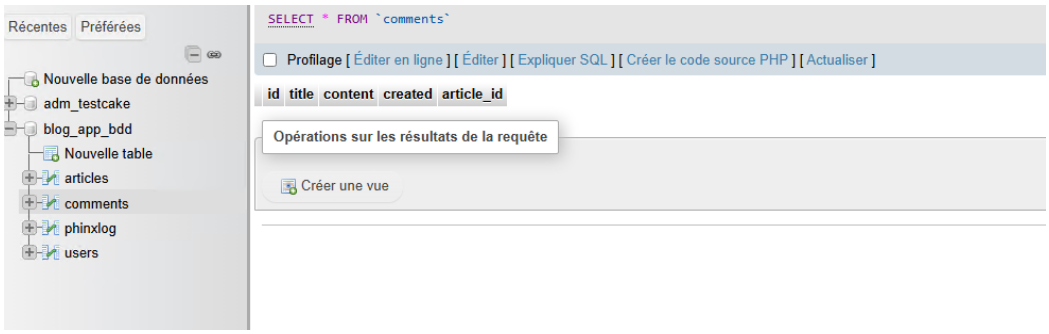
```
<td><?= count($user->articles) ?></td>
```

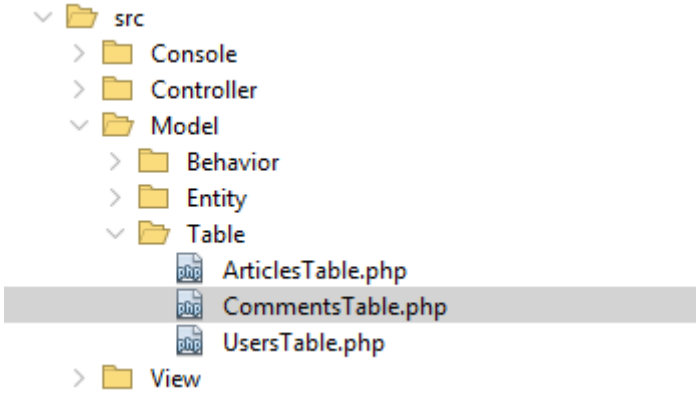
```php
        $table->addColumn('article_id', 'integer', [
            'null' => false,
        ]);
        $table->addForeignKey('article_id','articles','id');
        $table->create();
    }
```

```
C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251105042940 CreateComments: migrating
 == 20251105042940 CreateComments: migrated 1.1453s
All Done. Took 1.1967s
```

VII – Gestion des commentaires

- Créer la migration CreateComments :

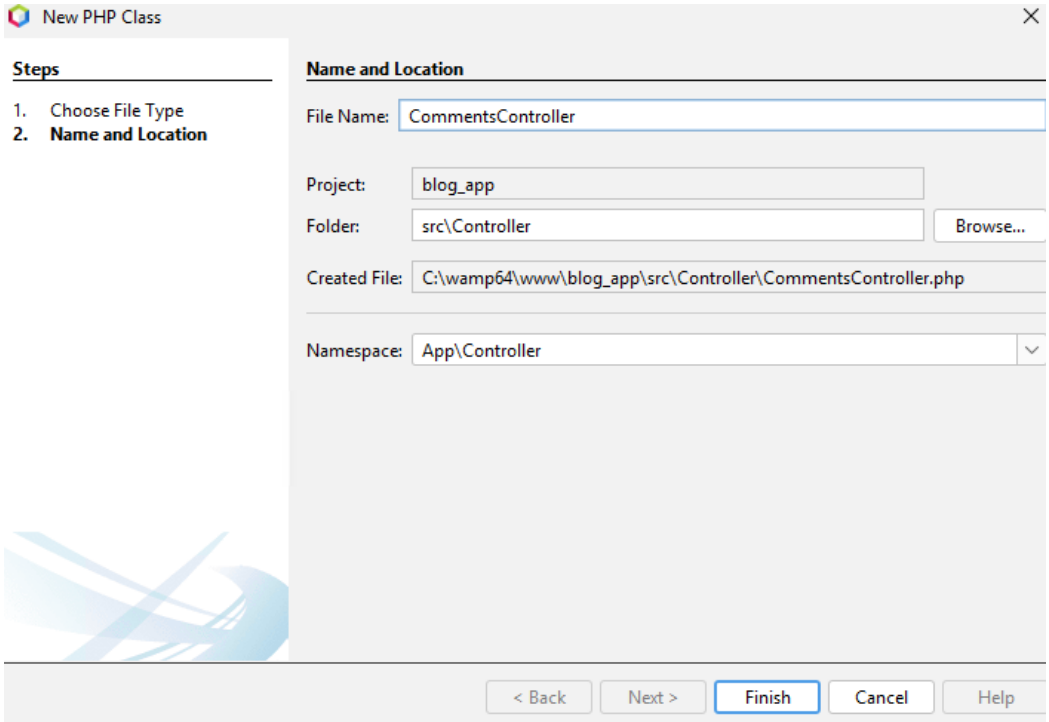





```php
<?php

namespace App\Model\Table;

use Cake\ORM\Table;
use Cake\Validation\Validator;

class CommentsTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Articles');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
        ->notEmptyString('title', __("Veuillez renseigner un title"))
        ->notEmptyString('content', __("Veuilez reseigner un content"))
        ->notEmptyString('article_id', __("Veuilez reseigner un article_id"));
        return $validator;
    }
}
```

VII.1 – Clé étrangère

- Dans la migration, ajouter la clé étrangère :




```
$table->addColumn('article_id', 'integer', ['null' => false]);
$table->addForeignKey('article_id', 'articles', 'id');
$table->create();
```

```
class ArticlesController extends AppController {

    public function index() {
        //on récupère tous les posts et on les stocke dans $mesArticles


        $mesArticles = $this->Articles->find()->contain([
                    'Users' => function ($q) {
                        return $q
                                ->select(['username', 'email']);
                    },
                    'Comments'])->all();
        $this->set(compact('mesArticles'));
    }
```

VII.2 – Création de la table comments

- Créer CommentsTable.php dans src/Model/Table :

```
        <th>Id</th>
        <th>Titre</th>
        <th>Nb commentaires</th>
        <th>Date de cr&eacute;ation</th>
        <th>Date de modification</th>
        <th>Créer par</th>
        <th>Action<th>
    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'affichage des
    <?php foreach ($mesArticles as $article): ?>
        <tr>
            <td><?= $article->id ?></td>
            <td>
                <?=
                $this->html->link($article->title, ['controller' => 'articles',
                    'action' => 'detail',
                    $article->id]); //l'url généré sera de la forme /articles/detail/…
                ?>
            </td>
            <td><?=count($article->comments) ?></td>
```

```
class CommentsTable extends Table {
    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Articles');
    }
    public function validationDefault(Validator $validator): Validator {
        $validator
            ->notEmptyString('title', __("Veuillez renseigner un title"))
            ->notEmptyString('content', __("Veuillez renseigner un content"))
            ->notEmptyString('article_id', __("Veuillez renseigner un article_id"));
        return $validator;
    }
}
```

```
class CommentsController extends AppController {

    public function add() {
        $leNewComment = $this->Comments->newEmptyEntity();
        if ($this->request->is('post')) {
            $leNewComment = $this->Comments->patchEntity($leNewComment, $this->request->getData());
            if ($this->Comments->save($leNewComment)) {
                $this->Flash->success(__("Le commentaire a été sauvegardé."));
                return $this->redirect(['action' => 'add']);
            } else
                $this->Flash->error(__("Impossible d'ajouter votre commentaire."));
        }
        $this->set(compact('leNewComment'));
    }
}
```

VII.3 – CommentsController

- Créer CommentsController.php
- Dans ArticlesController, mettre à jour index pour récupérer les commentaires :

```
<h1>Ajouter un commentaire</h1>
<?php
    echo $this->Form->create($leNewComment);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Sauvegarder le commentaire"));
    echo $this->Form->end();

?>
<?=
$this->html->link("retour a la liste des article", [
    'controller' => 'articles',
    'action' => 'index'],["class"=>"button"]);
```

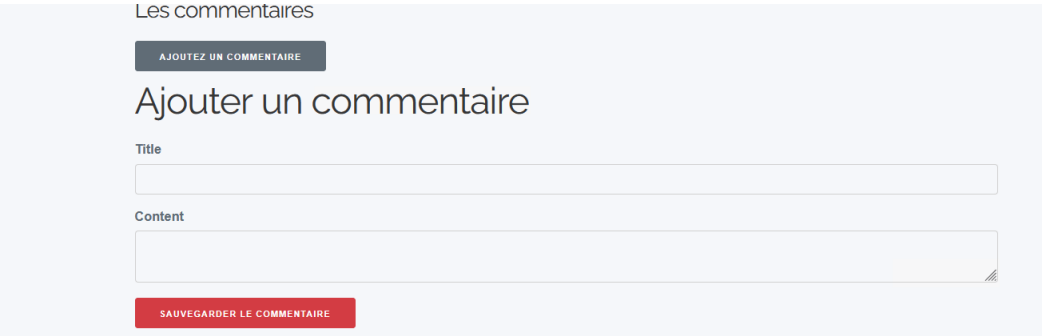
```
$mesArticles = $this->Articles->find()->contain([
    'Users' => function ($q) {
        return $q->select(['username', 'email']);
    },
    'Comments'
])->all();
```

```
public function add() {
    $leNewComment = $this->Comments->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewComment = $this->Comments->patchEntity($leNewComment, $this->request->getData());
        $leNewComment->article_id = 1;
        if ($this->Comments->save($leNewComment)) {
            $this->Flash->success(__("Le commentaire a été sauvegardé."));
            return $this->redirect(['action' => 'add']);
        } else
            $this->Flash->error(__("Impossible d'ajouter votre commentaire."));
    }
```

- Dans la vue index, afficher le nombre de commentaires :

```
<td><?=count($article->comments) ?></td>
```

```
echo $this->Html->script('jquery371min');
?>

<script>
    $(document).ready(function () {
        $("#showcom").click(function () {
            if ($("#display").is(":visible") == false)
            {
                $("#display").show();
            }
            else {
                $("#display").hide();
            }
        });
    });
</script>

<?=
$this->Html->link(
    'Ajoutez un commentaire', '#', ['class' => 'button', 'id' => 'showcom']
);
?>

<div id="display" style="display: none">
    <?= $this->element('comments'); ?>
</div>
```




```
public function detail($id = null) {
    $mesComments = $this->fetchTable('Comments');
        $leNewComment = $mesComments->newEmptyEntity();
```

- Dans CommentsController, ajouter la fonction add :

```
public function add() {
    $leNewComment = $this->Comments->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewComment = $this->Comments->patchEntity(
            $leNewComment, $this->request->getData());
        $leNewComment->article_id = 1;
        if ($this->Comments->save($leNewComment)) {
            $this->Flash->success(__("Le commentaire a été sauvegardé."));
            return $this->redirect(['action' => 'add']);
        } else {
            $this->Flash->error(__("Impossible d'ajouter votre commentaire."));
        }
    }
    $this->set(compact('leNewComment'));
}
```

```
public function detail($id = null) {
    $mesComments = $this->fetchTable('Comments');
        $leNewComment = $mesComments->newEmptyEntity();
        if ($this->request->is('post')) {
    $leNewComment = $mesComments->patchEntity($leNewComment, $this->request->getData());
    $leNewComment->article_id = $id;
    if ($mesComments->save($leNewComment)) {
        $this->Flash->success(__("Le commentaire a été sauvegardé."));
        return $this->redirect(['action' => 'detail',$id]);
    } else
        $this->Flash->error(__("Impossible d'ajouter votre commentaire."));
```

```
try {
    $leArticle = $this->Articles->get($id, contain: ['Comments'=> function ($q) {
                    return $q
                        ->order(['Comments.created asc']);
                }
                ]);
```

```
}
public function delete($id) {
    $this->request->allowMethod(['post', 'delete']);
    $leNewComment = $this->Comments->get($id);
    if ($this->Comments->delete($leNewComment)) {
        $this->Flash->success(__("Le post {0} d' id {1} a bien été supprimé ! ", $leNewComment->title, $leNewComment->id));
        return $this->redirect(['controller'=>'articles','action' => 'detail',$leNewComment->article_id]);
    }
}
```

```php
public function initialize(): void
{
    $this->loadComponent('Auth', [
        'loginRedirect' => [
            'controller' => 'Users',
            'action' => 'index'
        ],
        'authError' => 'Vous croyez vraiment que vous pouvez faire cela ?',
        'logoutRedirect' => [
            'controller' => 'Articles',
            'action' => 'index'
        ]
    ]);

    parent::initialize();

    $this->loadComponent('Flash');
```

```php
 */
public function beforeFilter(\Cake\Event\EventInterface $event) {
    $this->Auth->allow(['index', 'view']);
}
```

VII.4 – Création de la fonction add de commentaire

- Vue add de commentaire dans templates/Comments/add.php :

```php
public function beforeFilter(\Cake\Event\EventInterface $event)
{
    parent::beforeFilter($event);
    // Allow users to register and logout.
    // You should not add the "login" action to allow list. Doing so would
    // cause problems with normal functioning of AuthComponent.
    $this->Auth->allow(['add', 'logout']);
}

public function login()
{
    if ($this->request->is('post')) {
        $user = $this->Auth->identify();
        if ($user) {
            $this->Auth->setUser($user);
            return $this->redirect($this->Auth->redirectUrl());
        }
        $this->Flash->error(__('Mauvais username ou mot de passe, veuillez réessayer'));
    }
}

public function logout()
{
    return $this->redirect($this->Auth->logout());
}
```

```php
<h1>Ajouter un commentaire</h1>
<?php
    echo $this->Form->create($leNewComment);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Sauvegarder le commentaire"));
    echo $this->Form->end();
?>
```

```
C:\wamp64\www\blog_app>composer require cakephp/authentication
./composer.json has been updated
Running composer update cakephp/authentication
Loading composer repositories with package information
Updating dependencies
Lock file operations: 0 installs, 1 update, 0 removals
  - Upgrading cakephp/authentication (3.3.2 => 3.3.3)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 0 installs, 1 update, 0 removals
  - Downloading cakephp/authentication (3.3.3)
  - Upgrading cakephp/authentication (3.3.2 => 3.3.3): Extracting archive
Generating autoload files
58 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^3.3 for cakephp/authentication
```

- Modifier la fonction detail dans ArticlesController :

```
<div class="login-container">
    <div class="users form content">
        <div class="card">
            <div class="card-header">
                <h2 class="text-center"><?= __('Connexion') ?></h2>
            </div>
            <div class="card-body">
                <?= $this->Form->create() ?>
                <fieldset>
                    <legend class="text-muted text-center"><?= __('Veuillez saisir votre email et mot de passe') ?></legend>

                    <div class="form-group">
                        <?=
                        $this->Form->control('email', [
                            'label' => __('Email'),
                            'class' => 'form-control',
                            'placeholder' => __('votre@email.com')
                        ])
                        ?>
                    </div>

                    <div class="form-group">
                        <?=
                        $this->Form->control('password', [
                            'label' => __('Mot de passe'),
                            'class' => 'form-control',
                            'placeholder' => __('Votre mot de passe')
                        ])
                        ?>
                    </div>
                </fieldset>

                <div class="form-group text-center">

                    <?=
                    $this->Form->button(__('Se connecter'), [
                        'class' => 'btn btn-primary btn-lg btn-block'
                    ])
                    ?>
```

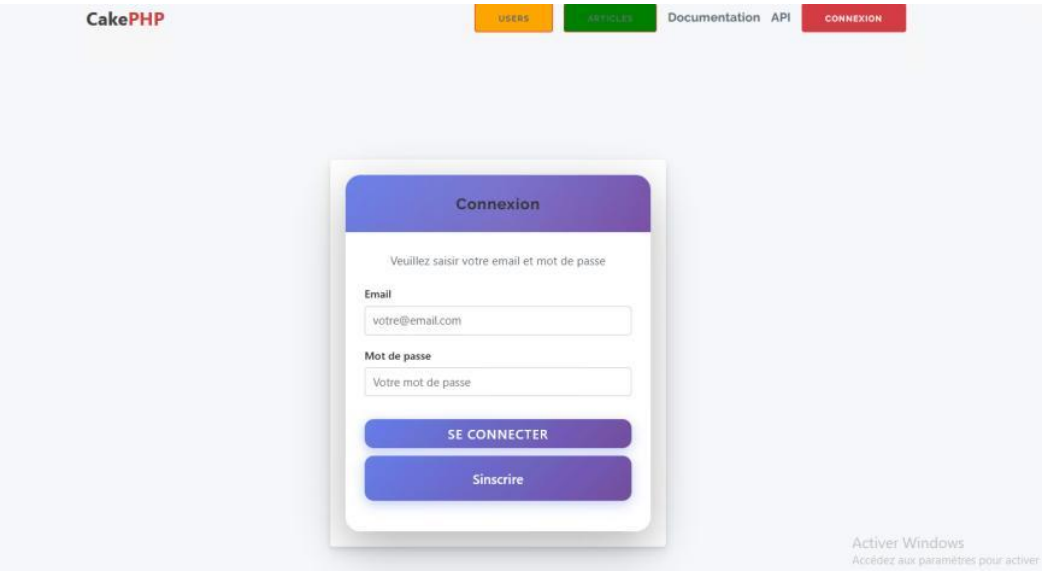
Activer Windows
Accédez aux paramètres pour activer Windows

```
public function detail($id = null) {
    $mesComments = $this->fetchTable('Comments');
    $leNewComment = $mesComments->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewComment = $mesComments->patchEntity(
            $leNewComment, $this->request->getData());
        $leNewComment->article_id = $id;
        if ($mesComments->save($leNewComment)) {
            $this->Flash->success(__("Le commentaire a été sauvegardé."));
            return $this->redirect(['action' => 'detail', $id]);
        } else {
            $this->Flash->error(__("Impossible d'ajouter votre commentaire."));
        }
    }
    $leArticle = $this->Articles->get($id, contain: ['Comments' => function ($q) {
        return $q->order(['Comments.created asc']);
    }]);
    $this->set(compact('leArticle', 'leNewComment'));
}
```

```
<?=
$this->html->link(__('Sinscrire'),
        ['controller' => 'users',
            'action' => 'add'], ['class' => 'btn btn-primary btn-lg btn-block','style' => 'color: white']
)
?>


<?= $this->Form->end() ?>
</div>
</div>
</div>
</div>
```

- Ajouter jQuery et le bouton d'affichage dans la vue detail :

```
echo $this->Html->script('jquery371min');
?>
<script>
    $(document).ready(function () {
        $("#showcom").click(function () {
            if ($("#display").is(":visible") == false) {
                $("#display").show();
            } else {
                $("#display").hide();
            }
        });
    });
</script>
<?= $this->Html->link('Ajoutez un commentaire', '#',
    ['class' => 'button', 'id' => 'showcom']); ?>
<div id="display" style="display: none">
    <?= $this->element('comments'); ?>
</div>
```




```
public function add() {
    $mesUsers = $this->fetchTable('Users')
            ->find('list', keyField: 'id', valueField: 'username')
            ->toArray();
    $leNewArticle = $this->Articles->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewArticle = $this->Articles->patchEntity($leNewArticle, $this->request->getData());
        $leNewArticle->user_id = $this->request->getAttribute("identity")->id;
        if ($this->Articles->save($leNewArticle)) {
            $this->Flash->success(__("L'article a été sauvegardé."));
            return $this->redirect(['action' => 'index']);
        } else
            $this->Flash->error(__("Impossible d'ajouter votre article."));
    }
    $this->set(compact('leNewArticle', 'mesUsers'));
}
```

```
$leArticle->user_id = $this->request->getAttribute("identity")->id;
```

```
class CommentsTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Articles');
        $this->belongsTo('Users');

    }
```

```
public function initialize(array $config): void {
    $this->addBehavior('Timestamp');
    $this->hasMany('Articles');
    $this->hasMany('Comments');
```

```
C:\wamp64\www\blog_app\bin>cake migrations create AlterCommentAddFkUsers

Creating file C:\wamp64\www\blog_app\config\Migrations\20251126045957_AlterCommentAddFkUsers.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251126045957_AlterCommentAddFkUsers.php`
```

VII.5 – Suppression d'un commentaire

```
*/
public function up(): void
{
    $table = $this->table('articles');
    $table->addColumn('user_id', 'integer', [
        'null' => false

    ]);
    $table->update();
    $this->execute('UPDATE articles SET user_id = (SELECT id FROM users LIMIT 1)');

    $table2 = $this->table('articles');
    $table2->addForeignKey('user_id','users','id');
    $table2->update();
}

public function down(): void
{
    $table = $this->table('articles');
    $table->dropForeignKey('user_id');
    $table->removeColumn('user_id');
    $table->update();
}
```

```
public function delete($id) {
    $this->request->allowMethod(['post', 'delete']);
    $leNewComment = $this->Comments->get($id);
    if ($this->Comments->delete($leNewComment)) {
        $this->Flash->success(__("Le post {0} d'id {1} a bien été supprimé !",
            $leNewComment->title, $leNewComment->id));
    }
    return $this->redirect(['controller' => 'articles',
        'action' => 'detail', $leNewComment->article_id]);
}
```

```
y {
  $leArticle = $this->Articles->get($id, contain: ['Comments.Users' => function ($q) {
          return $q
                  ->select(['Users.username'])
                  ->order(['Comments.created asc']);
      },
         'Users' => function ($q) {
              return $q
                      ->select(['Users.username']);
          }]);
```

```
<td><?= $article->user->username ?></td>
<?php
    $identity = $this->request->getAttribute('identity');
    if ($identity && ($article->user_id == $this->request->getAttribute('identity')->id)) { ?>
```

VIII – Authentification

- Installer le plugin :

```
composer require cakephp/authentication
```

```
$leArticle = $this->Articles->get($id);
$identity = $this->request->getAttribute('identity');
if ($identity && ($leArticle->user_id != $identity->id)){
    $this->Flash->error(__("Vous n'avez pas le droit d'éditer cet article"));
    return $this->redirect(['action' => 'index']);
```

- Dans AppController.php, ajouter la fonction initialize :

```
<?php

$identity = $this->request->getAttribute('identity');
if ($identity && ($comm->user_id == $identity->id || $leArticle->user_id == $identity->id)) {

    echo $this->Form->postLink(
            __('Supprimer'),
            ['controller' => 'comments', 'action' => 'delete', $comm->id],
            ['confirm' => __("Vraiment supprimer le commentaire ", $comm->title, $comm->id), 'class' => 'button']);
}
?>
```

```
public function initialize(): void {
    $this->loadComponent('Auth', [
        'loginRedirect' => [
            'controller' => 'Users', 'action' => 'index'
        ],
        'authError' => 'Vous croyez vraiment que vous pouvez faire cela ?',
        'logoutRedirect' => [
            'controller' => 'Articles', 'action' => 'index'
        ]
    ]);
    parent::initialize();
    $this->loadComponent('Flash');
}

public function beforeFilter(\Cake\Event\EventInterface $event) {
    $this->Auth->allow(['index', 'view']);
}
```

```
if ($this->request->getdata('tags_ids') != null) {
    //on recupere chaque id du tag
    foreach ($this->request->getdata('tags_ids') as $unTag_id) {
        //on crée l'enregistrement
        $articlestags = $this->fetchTable('ArticlesTags');
        $leNewTag = $articlestags->newEmptyEntity();
        //on associe au post le tag
        $leNewTag->article_id = $leNewArticle->id;
        $leNewTag->tag_id = $unTag_id;
        $leNewTag->priority = 5;
        //on enregistre dans la BDD
        $articlestags->save($leNewTag);
```

- Dans UsersController, ajouter les fonctions login et logout :

```
echo $this->Form->control('tags_ids', ['label' => 'Vous pouvez associer à votre post un ou plusieurs tags :', 'type' => 'select',
    'multiple'=>'checkbox',
'options' => $mesTags]);
echo $this->Form->control('priority');
```

```
public function beforeFilter(\Cake\Event\EventInterface $event) {
    parent::beforeFilter($event);
    $this->Auth->allow(['add', 'logout']);
}

public function login() {
    if ($this->request->is('post')) {
```

```
        $user = $this->Auth->identify();
        if ($user) {
            $this->Auth->setUser($user);
            return $this->redirect($this->Auth->redirectUrl());
        }
        $this->Flash->error(__("Mauvais username ou mot de passe, veuillez
réessayer"));
    }
}

public function logout() {
    return $this->redirect($this->Auth->logout());
}
```

```
public function index() {
 $query = $this->Tags->find();
    $tags = $this->paginate($this->Tags->find()->contain(['Articles']));

    $this->set(compact('tags'));

}
```

- Pour le login suivre la doc : https://book.cakephp.org/5/en/tutorials-and-examples/cms/authentication.html#adding-login

```
<table>
    <thead>
        <tr>
            <th><?= $this->Paginator->sort('id') ?></th>
            <th><?= $this->Paginator->sort('title') ?></th>
            <th><?= $this->Paginator->sort('nb articles') ?></th>
            <th><?= $this->Paginator->sort('created') ?></th>
            <th><?= $this->Paginator->sort('modified') ?></th>
            <th class="actions"><?= __('Actions') ?></th>
        </tr>
    </thead>
    <tbody>
        <?php foreach ($tags as $tag): ?>
            <tr>
                <td><?= $this->Number->format($tag->id) ?></td>
                <td><?= h($tag->title) ?></td>
                <td><?= h(count($tag->articles)) ?></td>
```

Tags

NEW TAG

Id	Title	Nb Articles	Created	Modified	Actions
3	cxvxvcx	1	11/28/25, 3:38 AM	12/5/25, 3:58 AM	View Edit Delete
6	sdfdsfsdf	0	12/3/25, 11:48 AM	12/5/25, 3:58 AM	View Edit Delete

< previous next >

Page 1 of 1, showing 2 record(s) out of 2 total

- Résultat sur le site :

```
<div style="margin: 20px 0;">
    <label style="display: block; margin-bottom: 15px; font-weight: bold;">Associer des tags :</label>

    <?php foreach ($mesTags as $id => $tagName): ?>
        <div style="border: 1px solid #ddd; padding: 12px; margin-bottom: 10px; background: #f9f9f9; border-radius: 4px;">
            <div style="display: flex; justify-content: space-between; align-items: stretch; min-height: 36px;">
                <div style="display: flex; align-items: center; flex: 1;">
                    <?=
                    $this->Form->checkbox("tags[_ids][]", [
                        'value' => $id,
                        'style' => 'margin: 0; margin-right: 10px; width: 18px; height: 18px; vertical-align: middle;',
                        'id' => "tag-{$id}",
                        'hiddenField' => false
                    ])
                    ?>
                    <?=
                    $this->Form->label("tag-{$id}", $tagName, [
                        'style' => 'font-weight: 600; cursor: pointer; margin: 0; line-height: 1.5;'
                    ])
                    ?>
                </div>

                <div style="display: flex; align-items: center; gap: 8px; margin: 0; padding: 0;">
                    <span style="font-size: 14px; color: #666; line-height: 1.5;">Priorité :</span>
                    <?=
                    $this->Form->number("tags[_joinData][{$id}][priority]", [
                        'id' => "tags[_joinData][{$id}][priority]",
                        'style' => 'width: 70px; height: 32px; padding: 4px 8px; border: 1px solid #ccc; border-radius: 4px; font-size: 1
                        'min' => 1,
                        'max' => 5,
                        'value' => 1,
                        'label' => false
                    ])
                    ?>
                </div>
```

Activer Windows

- Dans ArticlesController, lier le user_id lors de l'ajout :

```
// Dans la fonction add :
$leNewArticle->user_id = $this->request->getAttribute("identity")->id;

// Dans la fonction edit :
$leArticle->user_id = $this->request->getAttribute("identity")->id;
```

IX – Liens/association entre Users et Comments

IX.1 – Ajout des associations

- Dans CommentsTable.php, ajouter le belongsTo Users :

```
public function initialize(array $config): void {
    $this->addBehavior('Timestamp');
    $this->belongsTo('Articles');
    $this->belongsTo('Users');
}
```

- Dans UsersTable.php, ajouter le hasMany Comments :

```
y {
    $leArticle = $this->Articles->get($id,
            contain: ['Tags' => function ($q) {
                    return $q
                            ->select(['ArticlesTags.article_id',
                                'ArticlesTags.tag_id',
                                'ArticlesTags.priority']);
                }
    ]);
```

```
public function initialize(array $config): void {
    $this->addBehavior('Timestamp');
    $this->hasMany('Articles');
    $this->hasMany('Comments');
}
```

IX.2 – Ajout de la clé étrangère user_id

- Créer une migration AlterCommentAddFkUsers :

```
cake migrations create AlterCommentAddFkUsers
```

```
<div style = "margin: 20px 0;">
<label style = "display: block; margin-bottom: 15px; font-weight: bold;">Associer des tags :</label>

<?php foreach ($mesTags as $id => $tagName):
    ?>
    <div style="border: 1px solid #ddd; padding: 12px; margin-bottom: 10px; background: #f9f9f9; border-radius: 4px;">
        <div style="display: flex; justify-content: space-between; align-items: stretch; min-height: 36px;">
            <div style="display: flex; align-items: center; flex: 1;">
                <?=
                $this->Form->checkbox("tags[_ids][]", [
                    'value' => $id,
                    'style' => 'margin: 0; margin-right: 10px; width: 18px; height: 18px; vertical-align: middle;',
                    'id' => "tag-{$id}",
                    'hiddenField' => false
                ])
                ?>
                <?=
                $this->Form->label("tag-{$id}", $tagName, [
                    'style' => 'font-weight: 600; cursor: pointer; margin: 0; line-height: 1.5;'
                ])
                ?>
            </div>

            <div style="display: flex; align-items: center; gap: 8px; margin: 0; padding: 0;">
                <span style="font-size: 14px; color: #666; line-height: 1.5;">Priorité :</span>
                <?=
                $this->Form->number("tags[_joinData][{$id}][priority]", [
                    'id' => "tags[_joinData][{$id}][priority]",
                    'style' => 'width: 70px; height: 32px; padding: 4px 8px; border: 1px solid #ccc; border-radius: 4px; font-size: 14px;
                    'min' => 1,
                    'max' => 5,
                    'value' => 1,
```

Activer Windows

```php
public function up(): void {
    $table = $this->table('articles');
    $table->addColumn('user_id', 'integer', ['null' => false]);
    $table->update();
    $this->execute('UPDATE articles SET user_id = (SELECT id FROM users LIMIT 1)');
    $table2 = $this->table('articles');
    $table2->addForeignKey('user_id', 'users', 'id');
    $table2->update();
}

public function down(): void {
    $table = $this->table('articles');
    $table->dropForeignKey('user_id');
    $table->removeColumn('user_id');
    $table->update();
}
```

IX.3 – Ajout des jointures dans ArticlesController

- Pour faire une jointure dans une jointure, utiliser le contain imbriqué :

```php
$leNewComment->user_id = $this->request->getAttribute('identity')->get('id');

$leArticle = $this->Articles->get($id, contain: [
    'Comments.Users' => function ($q) {
        return $q->select(['Users.username'])
                 ->order(['Comments.created asc']);
    },
    'Users' => function ($q) {
        return $q->select(['Users.username']);
    }
]);
```

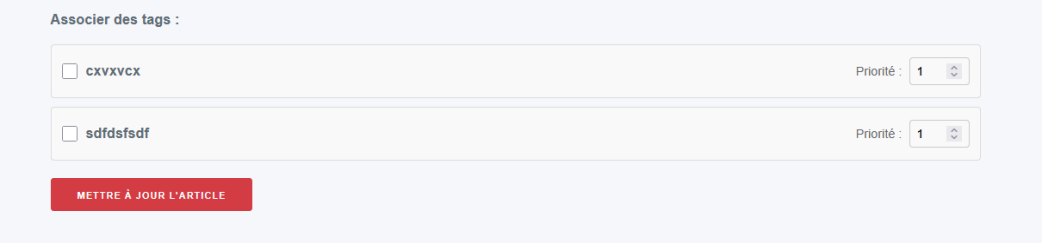
X – Sécuriser l'intégrité des données

X.1 – Disponibilité des boutons edit et supprimer

- Dans la vue index, conditionner l'affichage des boutons selon l'identité connectée :

```php
<div class="related">
    <h4><?= __('tags associé') ?></h4>
    <?php if (!empty($leArticle->tags)) : ?>
        <div class="table-responsive">
            <table>
                <tr>
                    <th><?= __('Id') ?></th>
                    <th><?= __('Title') ?></th>
                    <th><?= __('Modified') ?></th>
                    <th><?= __('priority') ?></th>
                    <th class="actions"><?= __('Actions') ?></th>
                </tr>
                <?php foreach ($leArticle->tags as $tag) : ?>
                    <tr>
                        <td><?= h($tag->ArticlesTags['tag_id']) ?></td>
                        <td><?= h($tag->title) ?></td>
                        <td><?= h($tag->modified->format(DATE_RFC850)) ?></td>
                        <td><?= h($tag->ArticlesTags['priority']) ?></td>
                        <td class="actions">
                            <?= $this->Html->link(__('View'), ['controller' => 'Tags', 'action' => 'view', $tag->ArticlesTags['tag_id']]
                            <?= $this->Html->link(__('Edit'), ['controller' => 'Tgas', 'action' => 'edit', $tag->ArticlesTags['tag_id']]
                            <?=
                            $this->Form->postLink(
                                    __('Delete'),
                                    ['controller' => 'Tags', 'action' => 'delete', $tag->ArticlesTags['tag_id']],
                                    [
                                        'method' => 'delete',
                                        'confirm' => __('Are you sure you want to delete # {0}?', $tag->ArticlesTags['tag_id']),
                                    ]
                            )
                            ?>
                        </td>
                    </tr>
```

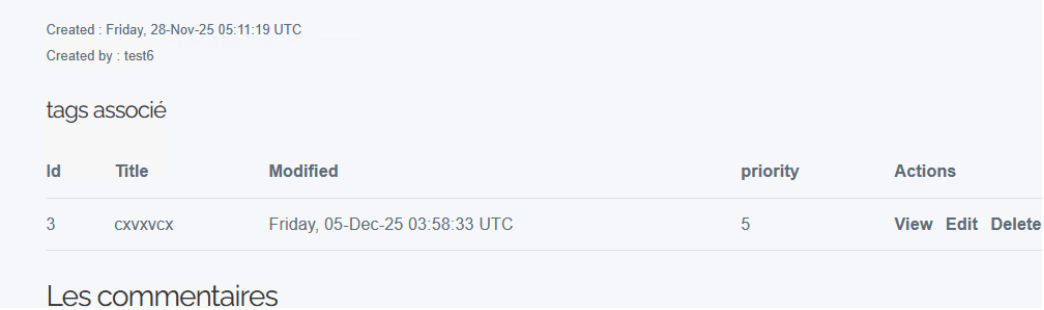
Activer Windows




```php
<?php
$identity = $this->request->getAttribute('identity');
if ($identity && ($article->user_id == $this->request->getAttribute('identity')->id)) { ?>
    // Boutons Edit et Supprimer ici
<?php } ?>
```

```php
                ])
                ?>
            </div>
        </div>
    </div>
<?php endforeach; ?>
<?php foreach ($leArticle->tags as $tag): ?>
        <script> document.getElementById('tag-<?= $tag->ArticlesTags['tag_id'] ?>').checked = true;</script>
        <script> document.getElementById('tags[_joinData][<?= $tag->ArticlesTags['tag_id']?>][priority]').value = "<?= $tag->ArticlesTag
    <?php endforeach; ?>
</div>
```

```php
'Tags' => function ($q) {
 return $q
        ->select(['ArticlesTags.tag_id','Tags.title','Tags.modified','ArticlesTags.priority']);
```



X.2 – Accessibilité de la page edit

- Dans la fonction edit du controller, vérifier que seul le créateur peut modifier :

```php
public function deleteAT($idTag = null, $idArt = null) {
    $this->request->allowMethod(['post', 'delete']);
    $articleTag = $this->ArticlesTags
            ->find()
            ->where(['tag_id' => $idTag, 'article_id' => $idArt])
            ->first()

    ;
    if ($this->ArticlesTags->delete($articleTag)) {
        $this->Flash->success(__('The articles tag has been deleted.'));
    } else {
        $this->Flash->error(__('The articles tag could not be deleted. Please, try again.'));
    }


    return $this->redirect( ['controller' => 'articles', 'action' => 'detail',$idArt]);
}
```

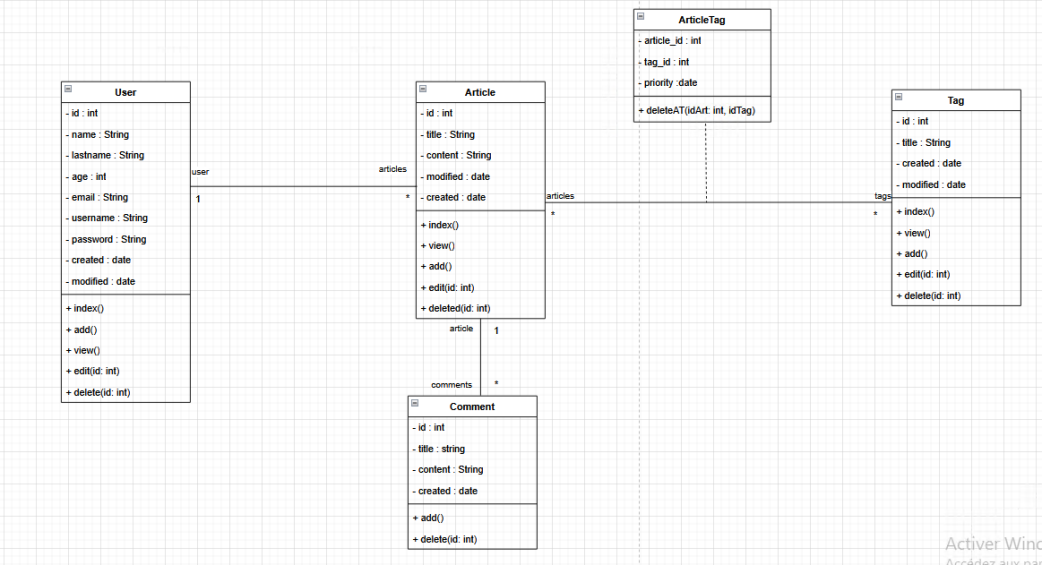
```php
$leArticle = $this->Articles->get($id);
$identity = $this->request->getAttribute('identity');
if ($identity && ($leArticle->user_id != $identity->id)) {
    $this->Flash->error(__("Vous n'avez pas le droit d'éditer cet article"));
    return $this->redirect(['action' => 'index']);
}
```

X.3 – Accessibilité du bouton supprimer des commentaires

- Dans la vue detail, conditionner le bouton supprimer :

```php
<?php if (count($leArticle->tags) == 0): ?>
    <h4><?= __("Aucun tag associé") ?></h4>
<?php else: ?>
    <h4><?= __('tags associé') ?></h4>
```

```php
<?php
$identity = $this->request->getAttribute('identity');
if ($identity && ($comm->user_id == $identity->id
        || $leArticle->user_id == $identity->id)) { ?>
    <?= $this->Form->postLink(__("Supprimer"),
        ['controller' => 'comments', 'action' => 'delete', $comm->id],
        ['confirm' => __("Vraiment supprimer le commentaire ?",
            $comm->title, $comm->id), 'class' => 'button']); ?>
<?php } ?>
```



XI – Gestion des Tags

XI.1 – Création fichier de migration

```
cake migrations create CreateArticlesTags
cake bake model articles_tags
cake bake all tags
```

- Dans la fonction add de l'ArticlesController, gérer l'ajout de tags :

```
if ($this->request->getdata('tags_ids') != null) {
    foreach ($this->request->getdata('tags_ids') as $unTag_id) {
        $articlestags = $this->fetchTable('ArticlesTags');
        $leNewTag = $articlestags->newEmptyEntity();
        $leNewTag->article_id = $leNewArticle->id;
        $leNewTag->tag_id = $unTag_id;
        $leNewTag->priority = 5;
        $articlestags->save($leNewTag);
    }
}
```

XI.2 – Ajouter nombre de tags dans index article

- Dans la vue index des articles :

```
<td><?= count($article->tags) ?></td>
```

- Dans ArticlesController, inclure les Tags dans le contain :

```
'Tags' => function ($q) {
    return $q->select(['id']);
}
```

XI.3 – Trier dans l'ordre décroissant les articles modifiés

- Dans ArticlesController :

```
])->orderDesc('Articles.modified')
->all();
```

XI.4 – Afficher le nombre d'article pour chaque tag

- Dans TagsController :

```
public function index() {
    $tags = $this->paginate(
        $this->Tags->find()->contain(['Articles']));
    $this->set(compact('tags'));
}
```

- Dans la vue index des tags :

```
<th><?= $this->Paginator->sort('nb articles') ?></th>
// ...
<td><?= h(count($tag->articles)) ?></td>
```

XI.5 – Ajouter les tags dans le add

- Vue add d'article – afficher les checkboxes pour choisir les tags :

```
<div style="margin: 30px 0;">
  <label style="display: block; margin-bottom: 15px; font-weight: bold;">Associer
des tags :</label>
  <?php foreach ($mesTags as $id => $tagName): ?>
    <div style="border: 1px solid #ddd; padding: 12px; margin-bottom: 10px;">
      <div style="display: flex; align-items: center; flex: 1;">
```

```
        <?=
        $this->Form->checkbox("tags_ids[]", [
            'value' => $id,
            'style' => 'margin: 0; margin-right: 10px; width: 18px; height: 18px;',
            'id' => "tag-{$id}",
            'hiddenField' => false
        ])
        ?>
        <?=
        $this->Form->label("tag-{$id}", $tagName, [
            'style' => 'font-weight: 600; cursor: pointer; margin: 0; line-height:
1.5;'
        ])
        ?>
      </div>
      <div style="display: flex; align-items: center; gap: 8px; padding: 0;">
        <span style="font-size: 14px; color: #666; line-height: 1.5;">Priorité
:</span>
        <?= $this->Form->number("tags_joinData[{$id}][priority]", [
            'id' => "tags_joinData[{$id}][priority]",
            'style' => 'width: 70px; height: 32px; padding: 4px 8px;',
            'min' => 1, 'max' => 5, 'value' => 1, 'label' => false
        ]) ?>
      </div>
    </div>
  <?php endforeach; ?>
</div>
echo $this->Form->control('priority');
```

XI.6 – Ajouter les tags dans le edit

- Dans la fonction edit de ArticlesController, inclure les tags existants :

```
$leArticle = $this->Articles->get($id, contain: [
    'Tags' => function ($q) {
        return $q->select([
            'ArticlesTags.article_id',
            'ArticlesTags.tag_id',
            'ArticlesTags.priority']);
    }
]);
```

- Vue edit d'article – pré-cocher les tags déjà associés :

```
<?php foreach ($mesTags as $id => $tagName): ?>
  // ... même structure que le add ...
  <?php endforeach; ?>
<?php foreach ($leArticle->tags as $tag): ?>
  <script>
    document.getElementById('tags[<?= $tag->ArticlesTags['tag_id'] ?>]').checked =
true;
    document.getElementById('tags_joinData[<?= $tag->ArticlesTags['tag_id']
?>][priority]')
      .value = <?= $tag->ArticlesTags['priority'] ?>;
  </script>
<?php endforeach; ?>
```

XI.7 – Afficher les tags dans le detail des articles

- Dans la fonction detail, inclure les tags dans le contain :

```
'Tags' => function ($q) {
    return $q->select([
```

```
        'ArticlesTags.tag_id', 'Tags.title',
        'Tags.modified', 'ArticlesTags.priority']);
}
```

- Dans la vue detail, afficher un tableau avec les tags :

```
<div class="related">
    <th><?= __("Tags associé") ?></th>
    <?php if (!empty($leArticle->tags)) : ?>
    <div class="table-responsive">
        <table>
            <tr>
                <th><?= __("Id") ?></th>
                <th><?= __("Title") ?></th>
                <th><?= __("Modified") ?></th>
                <th><?= __("Priority") ?></th>
                <th class="actions"><?= __("Actions") ?></th>
            </tr>
            <?php foreach ($leArticle->tags as $tag) : ?>
            <tr>
                <td><?= h($tag->ArticlesTags['tag_id']) ?></td>
                <td><?= h($tag->title) ?></td>
                <td><?= h($tag->modified->format(DATE_RFC850)) ?></td>
                <td><?= h($tag->ArticlesTags['priority']) ?></td>
                <td class="actions">
                    <?= $this->Html->link(__("View"), ['controller' => 'Tags',
'action' => 'view', $tag->ArticlesTags['tag_id']]) ?>
                    <?= $this->Html->link(__("Edit"), ['controller' => 'Tags',
'action' => 'edit', $tag->ArticlesTags['tag_id']]) ?>
                    <?= $this->Form->postLink(__("Delete"),
                        ['controller' => 'Tags', 'action' => 'delete', $tag-
>ArticlesTags['tag_id']],
                        ['confirm' => __("Are you sure ?", $tag-
>ArticlesTags['tag_id'])]) ?>
                </td>
            </tr>
            <?php endforeach; ?>
        </table>
    </div>
    <?php endif; ?>
</div>
```

XI.8 – Supprimer un tag d'un article

- Dans ArticlesTagsController, créer la fonction deleteAT :

```
public function deleteAT($idTag = null, $idArt = null) {
    $this->request->allowMethod(['post', 'delete']);
    $articleTag = $this->ArticlesTags
        ->find()
        ->where(['tag_id' => $idTag, 'article_id' => $idArt])
        ->first();
    if ($this->ArticlesTags->delete($articleTag)) {
        $this->Flash->success(__("The articles tag has been deleted."));
    } else {
        $this->Flash->error(__("The articles tag could not be deleted. Please, try
again."));
    }
    return $this->redirect(['controller' => 'articles', 'action' => 'detail',
$idArt]);
}
```

XI.9 – Afficher si des tags sont associées ou non

- Dans la vue detail d'articles, remplacer l'ancien if :

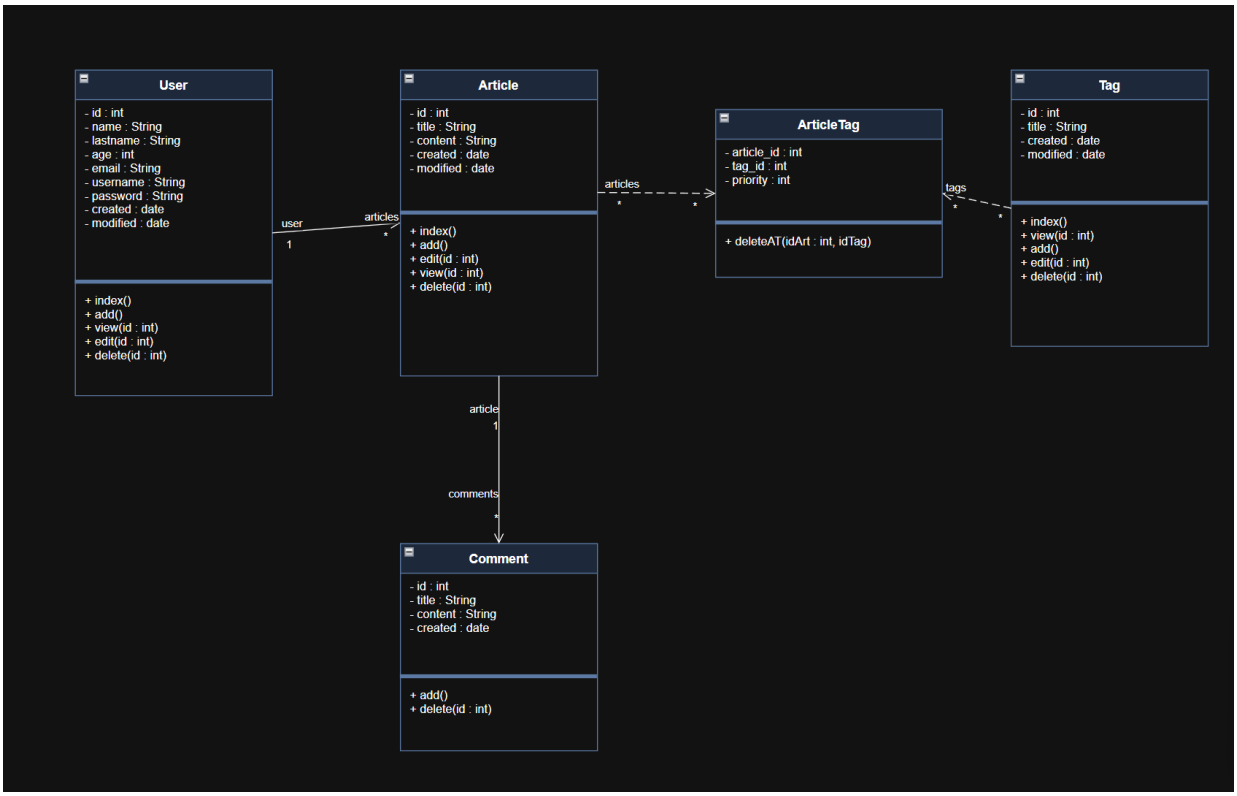
```
// Supprimer :
<?php if (!empty($leArticle->tags)) : ?>

// Remplacer par :
<?php if (count($leArticle->tags) == 0): ?>
    <h4><?= __("Aucun tag associé") ?></h4>
<?php else: ?>
    <h4><?= __('tags associé') ?></h4>
<?php endif; ?>
```

XII – Diagramme de classe

Le diagramme ci-dessous représente les relations entre les entités principales de l'application :

User	Article	Comment	Tag
- id : int	- id : int	- id : int	- id : int
- name : String	- title : String	- title : String	- title : String
- lastname : String	- content : String	- content : String	- created : date
- age : int	- created : date	- created : date	- modified : date
- email : String	- modified : date	- article_id : int	+ index()
- username : String	+ index()	- user_id : int	+ view()
- password : String	+ view()	+ add()	+ add()
+ index()	+ add()	+ delete(id: int)	+ edit(id: int)
+ add()	+ edit(id: int)		+ delete(id: int)
+ edit(id: int)	+ delete(id: int)		
+ delete(id: int)	ArticleTag: article_id, tag_id, priority, + deleteAT(idArt, idTag)		

Relations : User 1—< Articles | Article 1—< Comments | User 1—< Comments | Article >—< Tag (via ArticleTag)



— Fin de la documentation —
TERRAL Thomas – BTS SIO – 2025